SUP'COM
Higher School of Communication of Tunis

المدرسة العليا للمواصلات بتونس
ÉCOLE SUPÉRIEURE DES COMMUNICATIONS DE TUNIS

Telecommunications Engineering Training Cycle

Major:

Digital Services and Artificial Intelligence (SNI)

End of study internship REPORT

Topic :

Artificial Intelligence for Autonomous Driving using PiCar-V

Realized by :

Ramzi NAIFAR

Supervisors:

Mrs. Ferdaous CHAABENE
Ms. Yasmine KALLEL

Work proposed and realized in collaboration with:

Sagemcom Energie & Télécom

Sagemcom

Academic year: 2019/2020

HIGHER SCHOOL OF COMMUNICATION OF TUNIS

Abstract

End of Studies REPORT

Artificial Intelligence for Autonomous Driving using PiCar-V

by Ramzi NAIFAR

The self-driving car can be referred to as a robotic car having the ability of sensing its environment, navigating and mainly fulfilling the transportation needs while ensuring the safety and respecting the traffic code without the direct assistance of the human. It is a considerable step in the advancing future technology to test, apply and implement new artificial intelligence approaches that tackle the autonomous-driving problem.

In this context and in collaboration with SAGEMCOM Software & Technologies (SST), this report intends to outline the experimental process and result of implementing an AI-Based solution for self-driving cars, done as part of my graduation project. The object of this project is to develop a self-driving car system based on artificial intelligence using a PiCar-V. This latter will navigate, detect and classify people and traffic signs autonomously in real-time.

After studying the existing solutions and technologies that can be used, we developed two approaches: The first one is a basic self-driving system with rule-based control methods which means under manual intervention. The second one is the end-to-end self-learning method which learns to drive by itself.

Key-words : Autonomous Driving, Deep Learning, Deep Reinforcement Learning, Image Processing, Object Detection, Machine Learning, Raspberry Pi.

Résumé

La voiture autonome peut être dite robotique comme elle renferme un système de pilotage automatique. Elle permet de circuler sans intervention humaine dans des conditions de circulation réelles. Pour ce faire elle est équipée d'une série de capteurs et de caméras qui lui servent à modéliser son environnement en trois dimensions et à identifier les éléments qui le composent (marquage au sol, signalisation, bâtiments, véhicules, piétons...) afin qu'il puisse se guider en respectant les règles de circulation et éviter les obstacles. Cette véhicule promet d'améliorer la sécurité routière en réduisant le nombre d'accidents graves, fluidifier le traffic et libérer le conducteur du stress de route.

C'est dans ce contexte que s'inscrit mon projet de fin d'études en collaboration avec SAGEMCOM Software & Technologies. Ce rapport fournit une description de l'implémentation, la validation et le résultat de la mise en œuvre d'une solution basée sur l'IA pour les voitures autonomes en utilisant un PiCar-V.

Après avoir bien étudié l'état de l'art sur les technologies concernées ce projet a été traité avec deux approches. La première approche est basée sur un système de conduite autonome basique avec des méthodes de contrôle bien déterminées et nécessitant une intervention manuelle. La deuxième approche est la méthode d'auto-apprentissage de bout en bout qui apprend à conduire par elle-même.

Mots-clés : Apprentissage automatique, Apprentissage par renforcement profond, Apprentissage profond, Conduite autonome, Détection d'objets, Raspberry Pi, Traitement d'image.

Acknowledgements

It would be a great pleasure, before presenting my project, to express my gratitude to all those who supported me and provided me the possibility to complete this project. As well, I like to particularly show my recognition for SUP'COM staff and professors who afforded me an excellent education.

For this opportunity, I would like to express my sincere gratitude to Mrs Ferdaous CHAABANE, my academic supervisor, for helping me with the process of finding a suitable end of study project. I want to thank her for supporting me during this project by giving me feedback and valuable tips on how to handle and approach situations. I am also very appreciated to Mr Amine MEZGHANI for giving me the opportunity to join SAGEMCOM.

I would also like to thank Ms Yasmine KALLEL for her professionalism, continuous support, relevant remarks and contribution to providing me with the information necessary for this project.

As well, my deepest gratitude goes to all my colleagues who greatly inspired me to work in this project and all the interns that I worked and spent good moments with them.

Contents

Abstract **i**

Résumé **ii**

Acknowledgements **iii**

Contents **iv**

List of Figures **viii**

List of Tables **x**

List of Abbreviations **xi**

General Introduction **xii**

1 Project General Context **1**
1.1 Introduction 1
1.2 Host Company Presentation 1
1.2.1 SAGEMCOM Group 1
1.2.2 Activities 2
1.2.3 Organization 3
1.3 Project Scope 3
1.3.1 Project General Context 3
1.3.2 Problematic 4
1.3.3 Related Work 4
1.3.4 Proposed Solution 5
1.3.5 Project Objectives and Needs 5
1.4 Conclusion 5

2 Background Theory **7**
2.1 Introduction 7
2.2 Image Processing 7
2.2.1 Changing Color-spaces 7

2.2.2 Canny Edge Detection 8
2.2.3 Hough Line Transform 9
2.3 Deep Learning 9
2.3.1 Artificial Neural Network 9
2.3.2 Learning Process 10
2.3.3 Regularisation 12
2.3.4 Activation function 13
2.3.5 Batch Learning 14
2.3.6 Convolutional Neural Networks 14
Convolutional Layer 15
2.3.7 Pooling Layer 16
2.3.8 Example of application field: Object Detection 16
SSD MobileNet 16
Evaluation Metrics 17
2.4 Reinforcement Learning 20
2.4.1 Markov Decision Process 20
2.4.2 Discounted Expected Reward 21
2.4.3 Policy and Value Functions 22
2.4.4 Temporal Difference Methods 23
Q-Learning 24
2.5 Deep Reinforcement Learning (DRL) 24
2.5.1 Policy Gradient Methods 24
Actor-Critic Architecture 25
Proximal Policy Optimization (PPO) 25
Deep Deterministic Policy Gradient (DDPG) 27
Soft Actor-Critic (SAC) 27
2.6 Conclusion 28

3 Architecture and Working Environment **29**
3.1 Introduction 29
3.2 Working Environment 29
3.2.1 Hardware Tools 29
SunFounder Smart Video Car Kit V2.0(PiCar-V) 29
Intel Neural Compute Stick 2 31
Development Servers 31
3.2.2 Software Tools 32
Integrated Development Environment : 32
Libraries : 32

3.3 Simulation Environment . . . 33
3.4 Solution Architecture . . . 36
3.4.1 Global Architecture . . . 36
3.4.2 First Approach . . . 37
Lane Detection and Steering Control Method . . . 37
Traffic Sign and Pedestrian Detection . . . 37
Throttle Control . . . 38
3.4.3 Second Approach . . . 40
Data collection . . . 40
Sim2Real and State Representation Learning . . . 40
State representation Evaluation . . . 41
RL Evaluation . . . 41
3.5 Conclusion . . . 41

4 Implementation and Results **42**
4.1 Introduction . . . 42
4.2 First Approach . . . 42
4.2.1 Lane Detection and Steering Control Method . . . 42
Lane detection . . . 42
Steering Control . . . 45
4.2.2 Pedestrian and Traffic Sign Detection . . . 46
Data Collection . . . 46
Data Preprocessing . . . 47
Training and evaluation . . . 50
Deployment and Test . . . 52
4.2.3 Throttle Control Method . . . 52
4.3 Second Approach . . . 53
4.3.1 Data Collection . . . 54
4.3.2 Simulation to Reality and State Representation Learning . . . 54
4.3.3 RL-Agent Setup and training . . . 56
4.3.4 Evaluation . . . 58
4.4 Final Evaluation . . . 58
4.5 Conclusion . . . 59

Conclusion **60**

A Hardware configuration **61**
Raspberry Pi and NCS2 configuration . . . 61

B Lane detection parameters **62**

C RL setup and training **63**
Used Parameters in RL Algorithm 63
RL training results for lane navigation 64

D Additional equations **65**
Batch Normalisation 65
Bellman equation 66
Monte Carlo Method 66
Sarsa 67
Deep Q-Network (DQN) 67
REINFORCE algorithm 68

List of Figures

1.1 SAGEMCOM's footprint around the word 2
1.2 SST organization 3

2.1 RGB to HSV conversion 8
2.2 HSV colour space model 8
2.3 Image before and after application of Canny Edge Detection 8
2.4 Biological neuron vs. artificial neuron 10
2.5 Three-layer feedforward neural network and node of the network . . . 11
2.6 neuron with the function z = f(x, y, θ) 12
2.7 LeNet-5 Architecture 15
2.8 Example of max-pooling filter 16
2.9 SSD MobileNet Architecture 17
2.10 Precision and Recall 18
2.11 Intersection over union 19
2.12 The agent-environment interaction in a Markov decision process . . . 20
2.13 Example environment for RL 22
2.14 Actor-Critic Architecture 26

3.1 PiCar 30
3.2 Intel Neural Compute Stick 2 31
3.3 Construction and Play Mode 34
3.4 Donkey Simulator 35
3.5 New Updated Donkey Simulator 35
3.6 Global Architecture 36
3.7 Lane Detection and Steering Control Architecture 37
3.8 Steps for developing object detection method 38
3.9 Actions applied for each specific detected object 39
3.10 Steps for developing the second approach 40

4.1 Original image 42
4.2 Image in HSV colour 43
4.3 Green Area Mask 43
4.4 Edges of All Green Areas 43

4.5 Cropped Edges . . . 44
4.6 Lines segments . . . 44
4.7 Combination of Line Segments into Two Lines . . . 44
4.8 Heading direction . . . 45
4.9 Heading direction for one line detected . . . 45
4.10 Traffic Signs Data Examples . . . 47
4.11 Traffic Signs different class types Examples . . . 47
4.12 Pedestrians Lego Data Examples . . . 47
4.13 Filtered Traffic Signs . . . 48
4.14 Example of images after data augmentation . . . 48
4.15 Example of Data annotations . . . 49
4.16 Label map file example . . . 50
4.17 Total loss graph . . . 51
4.18 Example of validated test data . . . 51
4.19 False positive detected person . . . 51
4.20 Example of real test data . . . 52
4.21 Real robot navigation . . . 53
4.22 Example of simulated images . . . 54
4.23 Example of real images . . . 54
4.24 RL training graph evaluation for person crash avoidance . . . 57

C.1 RL training graph evaluation for lane navigation . . . 64

D.1 Concept of the Monte Carlo method . . . 67

List of Tables

4.1 Data identification 48
4.2 Data identification after data augmentation 49
4.3 Evaluation results 50
4.4 Throttle values for each detected object 53
4.5 Network architecture for the unsupervised image-to-image translation 56
4.6 Updated network architecture for the unsupervised image-to-image translation 57
4.7 RL training results for person crash avoidance 58
4.8 RL testing results for person crash avoidance 58
4.9 Evaluation results between first and second approach for lane detection 59
4.10 Evaluation results between first and second approach for person detection 59

B.1 Parameters applied in lane detection method 62

C.1 Common Parameters of Different RL Algorithms 63
C.2 SAC parameters 63
C.3 RL training results for lane navigation 64

List of Abbreviations

AI	**A**rtificial **I**ntelligence
ANN	**A**rtificial **N**eural **N**etwork
CNN	**C**onvolutional **N**eural **N**etwork
CPU	**C**entral **P**rocessing **U**nit
DL	**D**eep **L**earning
DNN	**D**eep **N**eural **N**etwork
DRL	**D**eep **R**einforcement **L**earning
GAN	**G**enerative **A**dversarial **N**etworks
GPU	**G**raphics **P**rocessing **U**nit
ML	**M**achine **L**earning
NCS2	**N**eural **C**ompute **S**tick **2**
RAM	**R**andom **A**ccess **M**emory
RL	**R**einforcement **L**earning
SDK	**S**oftware **D**evelopment **K**it
Sim2Real	**Sim**ulation to **Real**ity
VAE	**V**ariational **A**utoencoder
VPU	**V**ision **P**rocessing **U**nit

General Introduction

Over the last few years, the car industry has gone for a new orientation that more relies on the fourth industrial generation technologies such as Artificial Intelligence.

This orientation carries new promises for the car of the future that already began to roll out in foreign markets. Statistics reveal that the autonomous vehicle industry is growing globally by 16% each year, both established and new market intervenients like Tesla, Waymo, UBER, Nissan, Nvidia are involved in this product development.

This rush over providing the most performant self-driving car is justified by the countless benefits it insures, starting from a higher level of transportation's safety to better user experience.

Manufacturing of the most reliable autonomous car is set both as a priority for the variant industrial involved entities and a challenge for researchers in the AI field.

A self-driving car is defined as the assemblage of a specific set of hardware and software on a mechanical-based car that is capable of driving and navigating on the streets by sensing the environmental surroundings such as the lines of the road, traffic signs, and passengers. this car must take the decision on its own without direct human intervention.

Under these circumstances, The innovation group of SST has joined the race over developing new services for the car of the future within new approaches and adaptation for the AI technology. For this purpose, we propose the following work as a proof-of-concept of a possible AI-based solution for the self-driving car.

Mainly two approaches are put under experience:

- The first -and the most common one- is a "role-based control" solution that essentially uses a computer vision approach that performs the object detection through image processing and deep learning.

- The second one uses a reinforcement learning approach, that basically generates a model based on the maximization of the notion of cumulative reward.

In this report, we start with the first chapter, where we present the host company and the general concepts related to this project. In the second chapter, we introduce the theoretical background studies conducted within the scoop of this work. then in the third chapter, we present the project architecture and the working environment. Lastly, In the fourth chapter, we present the achieved through the obtained results while comparing the different implemented approaches.

Chapter 1

Project General Context

1.1 Introduction

It is essential before starting any work to identify the different environmental elements. Therefore, in this first chapter, an overview is provided about the host company Sagemcom Software & Technologies and the different areas of activities that it is involved in, followed by a brief description of the project context.

1.2 Host Company Presentation

1.2.1 SAGEMCOM Group

SAGEMCOM is a French group leader in high-value communications terminals on the European market. Its head office is located in Rueil-Malmaison, a French commune, the Group's nerve center where there is an R&D center with more than 500 dedicated employees. In 2014, the SAGEMCOM Group invests more than 75 million euros in its R&D [14]. With 4,000 employees in more than 50 countries, the group achieved sales of e2.05 billion, and more than 31.5 million terminals are designed, manufactured and delivered each year worldwide.

SAGEMCOM Software & Technologies (SST) is a subsidiary of the SAGEMCOM group which started its activities in July 2005. Its main mission is designing, developing and validating embedded software that meets broadband [1] communication market requirements. SST gathers in Tunisia approximately 300 development and validation engineers specialized in the field of communicating terminals with high added value. Figure 1.1 illustrates SAGEMCOM's footprint around the word.

[1]Broadband is wide bandwidth data transmission which transports multiple signals and traffic types.



FIGURE 1.1: SAGEMCOM's footprint around the word

1.2.2 Activities

Thanks to its common culture based on innovation, technological breakthroughs, high addedvalue and time to market, SST develops and produces a complete range of products that meet the markets in strong growth especially in three major markets:

Broadband : SAGEMCOM Broadband is one of Europe's leading manufacturers of Set-Top Boxes [2] and residential gateways. It offers products featuring the very latest technological breakthroughs, including STBs that are compatible with all the TV ecosystems, residential gateways offering end-to-end gigabit access, and Boxes combining broadband and TV access in a single product.

Smart City : SAGEMCOM boasts more than 50 years of experience in the design and deployment of telecommunications infrastructures projects (network infrastructures and information management), and more than 20 years in energy and smart metering (water, gas and electricity), by deploying secure and durable solutions.

Internet of Thing (IOT) : SAGEMCOM proposes an integrated end-to-end offer tailored for the Industrial Internet of Things , based on the open standard LoRaWAN [3] . This offer covers the radio RF [4] modules used to make the things connected, up to the entire infrastructure network allowing both the reception of data transmitted by the sensors, and control of these things by issuing commands from the network.

[2]STB is an information appliance device that displays output to a television set.

[3]LoRaWAN is a protocol for wide area networks. It is designed to allow low-powered devices to communicate with Internet-connected applications.

[4]Radio RF is any of the electromagnetic wave frequencies that lie in the range extending from below 3 kilohertz to about 300 gigahertz.

1.2.3 Organization

The organization of SST consists of an operation direction, a human resources service, general services, quality service, and the three following poles:

Decoders and Televisions activities pole : This pole includes many activities such as the design, development, and validation of STB embedded software, and the integration of access control bricks on the decoder.

Residential Terminals Activity pole : This pole includes many activities such as the design, development, and validation of embedded softwares for residential and professional gateways, offering VoIP services and TV, using xDSL and FTTH technologies.

Energy and Telecommunication activities pole : This pole is opened recently, where it ensures the conception, development, and validation of embedded software of digital electricity meters, and also assures the innovation for the internet of things based on the open standard LoRaWAN. Our internship is effectuated in this pole, within the innovation group which focuses on automotive activities (figure 1.2).

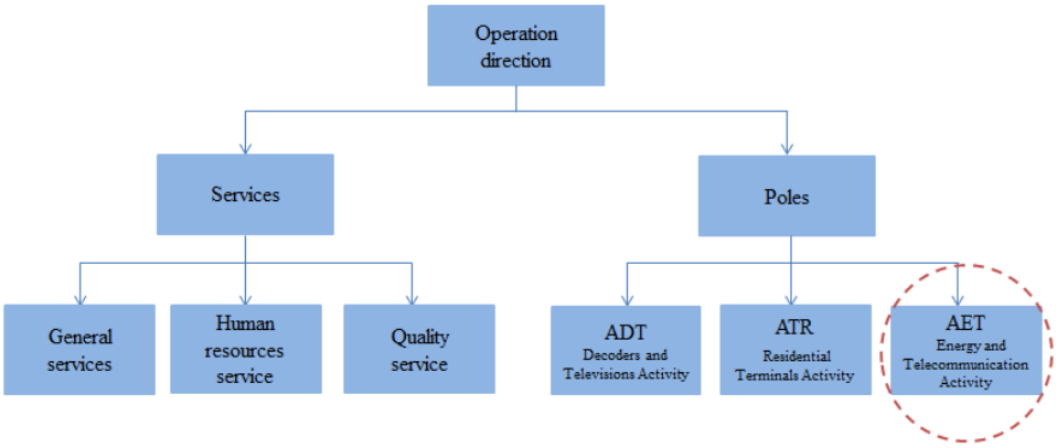


FIGURE 1.2: SST organization

1.3 Project Scope

1.3.1 Project General Context

In view of the diversity of its products and the wealth of its expertise and human resources, SAGEMCOM's ambition is to have a high level of technical, economic and social performance in order to maintain a lasting leadership in its field in a competitive environment. In order to enhance their competitiveness and develop their

productivity, SAGEMCOM has started attacking other domain related to Artificial Intelligence technology, particularly the autonomous driving.
In this context, SAGEMCOM proposed to develop a POC (Proof Of Concept) for autonomous driving offering new services and functionalities in order to find a better rate solution for the customer. This solution means lower costs and accident rate so more secure for customer and even for pedestrian.

1.3.2 Problematic

Today's car manufacturers are aiming to make the driving experience easier, safer and autonomous all of that is becoming more tangible thanks to the present days' technologies. To keep the passengers' safety, any accident is unacceptable. Therefore, the reliability and security must satisfy the stringent standard. The accuracy and robustness of the sensors and algorithms are required extreme precision in the process of self-driving vehicles. On the other hand, self-driving cars are products for average consumers, so the cost of cars need to be controlled. High-precision sensors can improve the accuracy of the algorithms but very expensive. This is a difficult contradiction that need to be solved. Add to this, the problem of designing a fully autonomous car, still unsolved yet as the requirements are not met enough with the different proposed solution. thus researchers still investigating and proposing more efficient solutions that try to fulfil those standards.

1.3.3 Related Work

Our aim is investigating a safe self-driving car. Right now, the most common methods to train the car to perform self-driving are behavioural cloning and line following. On a high level, behavioural cloning works by using a convolutional neural network (Chapter 2.3.6) to learn a mapping between car images (collected human driving data through the front camera) and steering angle and throttle values through supervised learning. The other method, line-following, works by using image processing (Chapter 2.2) and object detection techniques to track the middle line to get the car to follow the line and act accordingly to objects with rule-based control method. However, these are the capabilities that under manual intervention. We hope that cars can learn to drive by themselves, which is an intelligent way.
Recently, the use of virtual simulation techniques to train intensive learning models and then migrated to reality has been largely verified. OpenAI has developed a robotic arm called Dactyl [17] that trains AI robots in a virtual environment and finally applies them to physical robots. From the above work, we can see that many

of the visual autopilot algorithms learn through the neural network under the condition of supervised learning, get the mapping relationship, and then control. But this is not smart enough. Tesla's driverless accident [7] is caused by perceived module failure in a bright light environment. Reinforcement learning (Chapter 2.4) can do so, even in the event of failure of certain modules. Reinforcement learning makes it easier to learn a range of behaviours. Automated driving requires a series of corrective actions to drive successfully. If only the data is annotated, the learned model is offset a bit at a time and at the end, it may be offset a lot, with devastating consequences. Reinforcement learning can learn to automatically correct the offset. The key to a truly autonomous vehicle is self-learning, and using more sensors does not solve the problem. In this case, we use the algorithm of deep reinforcement learning to make a POC of a self-driving car.

1.3.4 Proposed Solution

Firstly the proposed solution's sensor is only the camera so we can resolve the cost problem, then we must verify this solution by trying different approaches to find the most optimal one. To do so, we will use a small robot car for all the experience as it is a safe and cheap solution for first time experience. The first approach is the rule-based control method, and the second is the RL method so we can prove that the theory of self-learning is the most optimal one.

1.3.5 Project Objectives and Needs

In this project, as we specified above, the main objective is to develop a safe self-driving system using a small robot car. In this case, a deep study of the state of the art is needed to define some use cases that will be used to fulfil the global objective of the project. The objectives of the project are essentially:

- Develop and implement the most known and used approach into the robot car, so in the final, this latter can detect lines, pedestrian and four different types of traffic signs to navigate accordingly in real-time.
- Try to adjust the previous approach, so it is well improved or implements a new one performing a better accuracy..

1.4 Conclusion

This chapter was dedicated to describe the host company and the project context as an introductory step to help better understand and get to the made work. A state of

the art, the object of the next chapter, is essential to define the main axes of the project then to extract the main functionalities to satisfy.

Chapter 2

Background Theory

2.1 Introduction

This chapter summarizes the relevant fundamentals for further understanding of the topic of the proposed work. In chapter 2.2, basic concepts of Image Processing are presented. Then, in Chapters 2.3, 2.4 and 2.5 we expose briefly the main bases of DL, RL and DRL to understand which type of models are the most suitable to fit the project's specification.

2.2 Image Processing

Image processing is an umbrella term for many functions that analyze images or convert one representation of an image into another. Although certain kinds of analog processing were performed in the past, today image processing is done in the digital domain. It is used in many fields, including surveillance, medical imaging, machine vision, robotics, autonomous-driving, computer-generated imagery, videoconferencing, satellite data analysis, etc.
We will explore some of the existing methods used to deal with digital images starting by a very basic approach of Changing Color-spaces. As a more advanced level of interpretation comes the Canny Edge Detection and Hough Line Transform.

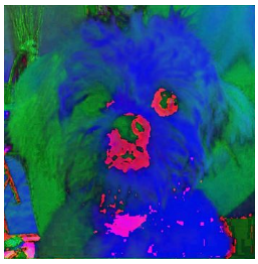
2.2.1 Changing Color-spaces

Colour space is a mode that colours are combined in such a way they can represent the input image. There are many colour-spaces conversion methods such us RGB $\leftrightarrow$ HSV which is the most widely used. Figure 2.1 represent a conversion from RGB image to HSV image.
RGB (Red/Green/Blue) is the standard colour space that we know, for example, the real captured images by the camera represented in RGB mode. HSV (Hue / Saturation / Value) is an alternative representation of the RGB colour model, which used

to help the process of extraction of the relevant wanted colour from images. HSV describes colours in terms of their shade (Hue), their amount of grey (Saturation) and their brightness (Value) (Figure 2.2).

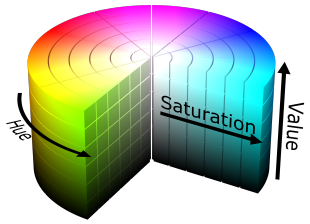


(A) Original RGB image

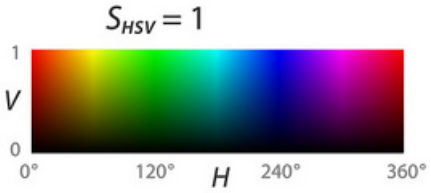


(B) HSV image version

FIGURE 2.1: RGB to HSV conversion



(A)



(B)

FIGURE 2.2: HSV colour space model

2.2.2 Canny Edge Detection

The Canny edge detector is an edge detection operator that uses a multi-stage algorithm to detect a wide range of edges in images.
It uses a filter based on the derivative of a Gaussian in order to compute the intensity of the gradients. The Gaussian reduces the effect of noise present in the image. Then, potential edges are thinned down to 1-pixel curves. Finally, edge pixels are kept or removed using hysteresis thresholding on the gradient magnitude (Figure 2.3).

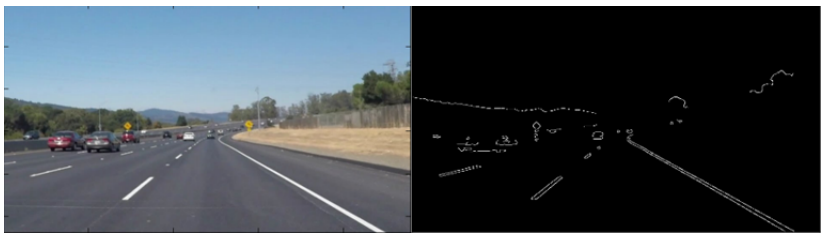


FIGURE 2.3: Image before and after application of Canny Edge Detection

2.2.3 Hough Line Transform

Hough Line Transform is a popular technique to detect any shape if it can represented in mathematical form. It can detect the shape even if it is broken or distorted a little bit. A line can be represented as y = mx+c or in parametric form, as $\rho = x\cos\theta + y\sin\theta$ where ρ is the perpendicular distance from origin to the line, and θ is the angle formed by this perpendicular line and horizontal axis measured in counter-clockwise.

So if line is passing below the origin, it will have a positive ρ and angle less than 180. If it is going above the origin, instead of taking angle greater than 180, angle is taken less than 180, and ρ is taken negative. Any vertical line will have 0 degree and horizontal lines will have 90 degree.

Any line can be represented in these two terms, (ρ, θ). So first it creates a 2D array or accumulator (to hold values of two parameters) and it is set to 0 initially. Rows denoted ρ and columns denoted θ. Size of array depends on the accuracy we need. If we want the accuracy of angles to be 1 degree, then we need 180 columns. For ρ, the maximum distance possible is the diagonal length of the image. So taking one pixel accuracy, number of rows can be diagonal length of the image.

2.3 Deep Learning

Deep learning is a technique for implementing ML built on a deep hierarchy of interconnected neural network layers. It takes massive amounts of data and determines the common rules and features associated with the data.
So to understand what is really deep learning we must at first introduce the ANN fallowed by the types of different layers and techniques applied to achieve better training results. Finally, we will have an overview of some algorithms applied in this project.

2.3.1 Artificial Neural Network

The artificial neuron is inspired by the biological neuron from the animal brain. The human brain consists of approximately 100 billion neurons to process sensory information like vision, touch, and acoustics. A single neuron has several inputs – called dendrites – coming from other preceding neurons. The neuron processes the inputs and if a certain action potential is reached, the neuron "fires" through its single output – called axon. The output of the axon will be forwarded to all following connected neurons. An artificial neuron (also called perceptron) models the biological neuron

in a simplified way. Each artificial neuron has n input connections. The neuron processes the inputs by taking the weighted sum, adding a bias b and applying an activation function(will be discovered later): $f(\sum_{i=0}^{n} \theta_i x_i + b)$. The perceptron composed of layers: input layer(initial data for the neural network it can be the output of an other perceptron), hidden layer (intermediate layer between input and output layer and place where all the computation is done) and output layer. Figure 2.4 illustrates the parallels of a biological and an artificial neuron.

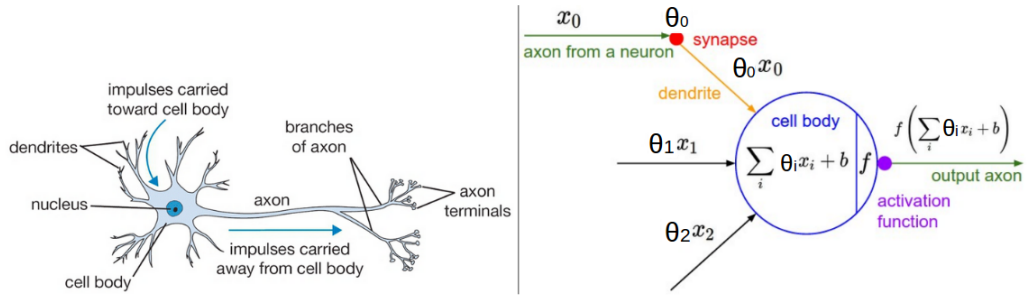
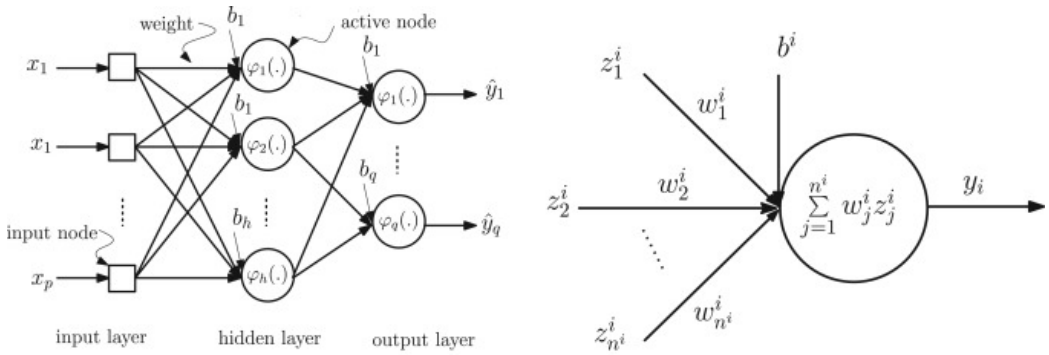


FIGURE 2.4: Biological neuron (left) vs. artificial neuron (right). The artificial neuron models the dendrites as weighted inputs and processes the sum through an activation function $f(\sum_{i=0}^{n} \theta_i x_i + b)$ [29]

Neural networks approximate a non-linear function f(x) by composing multiple neurons in a chain. The parameter set θ that contains all weights θ_i of all neurons needs to be adjusted in such a manner that the Neural Network results in the best possible function approximation. The process of finding a good parameter set θ is called learning. A feedforward Neural Network organizes the artificial neurons in different layers. The neurons in the layers are connected to each other in a forwarding manner. There are no connections that are fed back to previous neurons. Each network has one input layer, that processes the raw input data, and one output layer, that contains the approximated result. In between those two layers can be one or more hidden layers where the relevant computing is happening. Figure 2.5 shows a Fully-connected, feedforward Deep Neural Network. It is fully-connected because all neurons of the outgoing layer are connected to all neurons of the incoming layer. This is not absolutely necessary [8].

2.3.2 Learning Process

The goal of the learning process is to find a parameter set that results in the best possible function approximation. In supervised learning, the true output Y of a certain input X is given and can be used to update the parameters . It is an iterative process, consisting of the following steps.



(a) Three-layer feedforward neural network (b) Node of the network

FIGURE 2.5: Three-layer feedforward neural network and node of the network

- **Forward Pass :** The input X is forwarded through the network and one gets the predicted output Ypred = f(X, θ).

- **Loss :** The predicted output Ypred is compared to the true output Y by computing the loss L(θ). The choice of the loss function depends on the learning task. In the following, relevant loss functions are listed:
  - Mean-square-error [21]. It is widely used and computes the L2-distance between Ypred and Y.

$$L(\theta) = \frac{1}{2}\sum_{i=1}^{n}(Y_i - Y_{pred,i})^2 \tag{2.1}$$

  – Logistic Loss function [21]. The logistic loss function punishes points that are classified correctly with a low confidence. Still wrongly classified samples are punished more strongly.

$$L(\theta) = \sum_{i=1}^{n} log(1 + exp(-Y_i Y_{pred,i})) \tag{2.2}$$

- **Back-propagation :** The global gradient of loss $\nabla L(\theta)$ is computed and back-propagated through the network. The back-propagation algorithm, provides local gradient of loss to all hidden neurons. The algorithms underlying concept is the chain rule. The chain rule computes derivatives of composed function by multiplying local derivatives. Given the function y = g(x) and z = f(g(x)), the derivative $\frac{\partial z}{\partial x}$ can be computed according to equation 2.3 [8].

$$\frac{\partial z}{\partial x} = \frac{\partial z}{\partial y}\frac{\partial y}{\partial x} \tag{2.3}$$

The chain rule is used to propagate the global gradient loss $\frac{\partial L(\theta)}{\partial \theta}$ back through the network. It flows in the opposite direction of the forward pass. Figure 2.6 shows a neuron with the function z = f(x, y, θ). Its local derivatives are $\frac{\partial z}{\partial x}$, $\frac{\partial z}{\partial y}$ and can be determined during the forward pass. The local gradient of loss is computed during back-propagation by multiplying the local derivative with the local gradient of loss of the connected neuron of the next layer $\frac{\partial L}{\partial z}$. As result, one gets a local gradient of loss for each input of the neuron: $\frac{\partial L}{\partial x}$, $\frac{\partial L}{\partial y}$. They will be further back-propagated to the other preceding neurons. In case a neuron is connected to several neurons in the next layer, the gradients are simply added up [12].

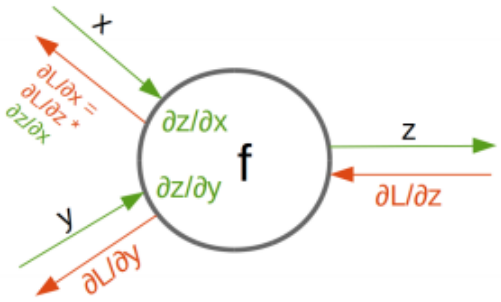


FIGURE 2.6: Neuron with the function z = f(x, y, θ)

- **Update :** The weights of all neurons are updated. A common optimizer is stochastic gradient descent (SGD). It combines Batch Learning from section 2.3.5 with gradient descent. Gradient descent changes the weights in the negative direction of the gradient of loss so that the function approximation approaches closer to the minimum in each iteration. It is expected that after a number of iterations, a local minimum is reached. Equation 2.4 shows the corresponding update rule of gradient descent. α is the learning rate parameter that defines how quickly the minimum should be approached. If the learning rate α is too high, there is a risk that the minimum cannot be reached, because the taken steps are too big and will be overshot.

$$\theta_i \leftarrow \theta_i + \alpha \nabla_\theta L(\theta) \tag{2.4}$$

2.3.3 Regularisation

The final goal of learning is that the function approximation generalizes over the presented data, i.e., it shows similar performance on new, unseen data. A good compromise between under- and overfitting needs to be found. Underfitting occurs when

the approximated function is too simple. It generalizes on the data, but the prediction error is too high for all data points. Underfitting often results from insufficient, small training data sets with a lack of diversity. Overfitting describes the contrary: the approximated function is too complex. It represents the training points really well, but unseen points are predicted poorly. Regularization is an approach to prevent overfitting. The loss function will be extended with a regularization term $\Omega(\theta)$, that tries to keep the approximated function as simple as possible. Equation 2.5 shows the extended regularized objective loss function $\tilde{L}$. λ is the regularization factor, that weighs the regularization term $\Omega(\theta)$ against the original loss function. If $\lambda = 0$, there is no regularization [8].

$$\tilde{L}(\theta) = L(\theta, Y, Y_{pred}) + \lambda\Omega(\theta) \tag{2.5}$$

Equation 2.6 shows the L^2-regularization. It adds up the squared sum of the weights θ to the objective loss function. The regularization method aims to keep the weights small.

$$\tilde{L}(\theta) = L(\theta, Y, Y_{pred}) + \lambda\frac{1}{2}\|\theta\|^2 \tag{2.6}$$

Equation 2.7 shows the L^1-regularization. In contrary to the L^2-regularization, the weights are only punished linearly. Large weights are not punished stronger, so that it is possible to have large weights if at the same time several small weights get zero. The regularization method leads to a sparser solution [8].

$$\tilde{L}(\theta) = L(\theta, Y, Y_{pred}) + \lambda\frac{1}{2}\|\theta\| \tag{2.7}$$

2.3.4 Activation function

There are three different commonly used activation functions: sigmoid, tanh and ReLU, that will be discussed in this chapter.

The sigmoid function is shown in equation 2.8. It maps the input value x between the range of 0 and 1. Large negative values become 0 and large positive values become 1.

$$sigm(x) = \frac{1}{1 + \exp(-x)} \tag{2.8}$$

The sigmoid function is less used because it has some crucial disadvantages. If the neuron's output saturates at 0 or 1, the local gradient gets almost zero. At backpropagation, the global gradient will be multiplied with the local gradients, so that the product ends up zero as well. Eventually, the weights will not change, and the

network is not capable of learning effectively. It is especially problematic if the network is initialized with weights that directly end up in saturating outputs. Another disadvantage is that the output of the sigmoid function is not zero-centered [12].

The tanh function is shown in equation 2.9. It zero-centers the sigmoid function. Still the disadvantage of saturation remains.

$$tanh(x) = 2sigm(2x) - 1 \tag{2.9}$$

The ReLU function is the most popular function and shown in equation 2.10. It does not allow the output to get smaller than zero. It has a non-saturating form and allows the gradient to converge faster during training. Another advantage is that it is a simple function with a small computational cost [12].

$$ReLU(x) = max(0, x) \tag{2.10}$$

2.3.5 Batch Learning

The idea of Batch Learning is to process a set of m training samples (mini-batches) instead of just a single training example. The gradient is averaged over all m processed training examples. This can lead to a more accurate gradient with less variance, resulting in reduced training time. Besides, Batch Learning speeds up the training when using a graphics processing unit (GPU). All training samples can be processed independently, i.e., in parallel [12].

2.3.6 Convolutional Neural Networks

Convolutional Neural Networks are inspired by the receptive field in the brain, that processes sensor input data and is sensitive to certain stimuli, e.g., edges in the visual system. They handle large input data efficiently and are consequently widely used in state-of-the art approaches in the fields of Computer Vision, like e.g. object detection or image segmentation.

Figure 2.7 shows the LeNet-5 [13] that recognizes digits in images. It provides a typical architecture of Convolutional Neural Networks, consisting of stacks of Convolutional Layers, followed by a subsampling Pooling Layer. The final hidden layers of the network are normally fully-connected to compute the final low-dimensional output of the network. It can be assumed, that in the early stages of the network low-level features like edges and corners are learned, while in later layers those features are combined to high-level features.

Convolutional Layer

The Convolutional Layer builds on the discrete convolution operation, that applies a square filter f of the size [m $\times$ m] with m = 2k + 1 to an input matrix g at position [x, y] by computing the dot product.

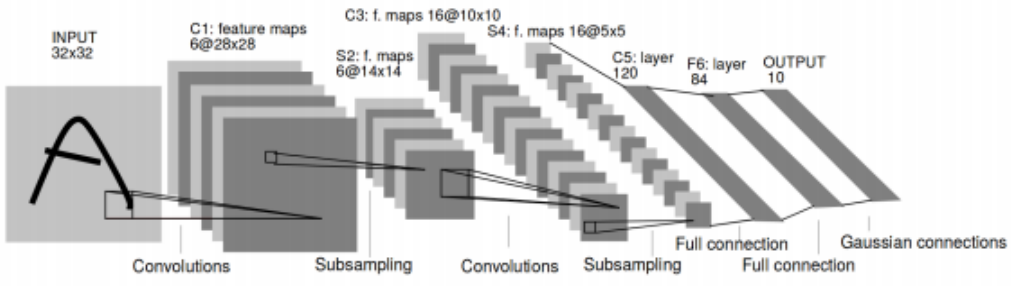


FIGURE 2.7: LeNet-5 [13] Architecture

One neuron in a Convolutional Layer is represented by a filter of the size [m $\times$ m $\times$ d]. The weights of the neuron are the filter values as well as a bias b. The filter will be shifted over the input matrix g with depth d and produces an output h[x, y] for each position [x, y]. Note that the input matrix g and the filter f have the same depth d. To produce an output h that has the same size as the input g, zero padding can be applied. Zero padding extends the input matrix g by (m - 1)/2 rows or columns with zero-values on each side. A set of different filters (= neurons) forms the Convolutional Layer. All filters have the same size, but different filter values, and are all applied to the same input g, producing a so called activation map. The final output of the layer is a stack of all activation maps.

It is common to shift the filter with a constant stride S over the input so that every Sth position of the input will be convolved. It results in a reduction of the data size in the next layer.

Compared to a Fully-connected Layer, the number of weights in a Convolutional Layer is kept small and the computation in the layer is more efficient. Generally, the filter size is kept much smaller than the input data, leading to the detection of small, lowlevel features. Small filters require fewer parameters as well as fewer operations in the convolution operation. In addition, the filter is applied to the different locations of the input due to filter shifting. The intuition behind this is that the same features can appear at different locations of the input and can be detected by the same neuron. For this reason, feature detection with Convolutional Layers is invariant in translation [8].

2.3.7 Pooling Layer

The Pooling Layer has a subsampling function in the spatial dimensions width and height by applying a downsampling filter to the input. Common pooling filters are max- and average-pooling. At max-pooling, a filter of size [m $\times$ m] slides over the input and only the maximum value remains in the output. Figure 2.8 provides an example: A [2 $\times$ 2]-filter is shifted over the 2-dimensional input with a stride of 2. The resulting output size is a quarter of the input size. In average-pooling the average of each position [x, y] and its neighbors is computed by the filter.

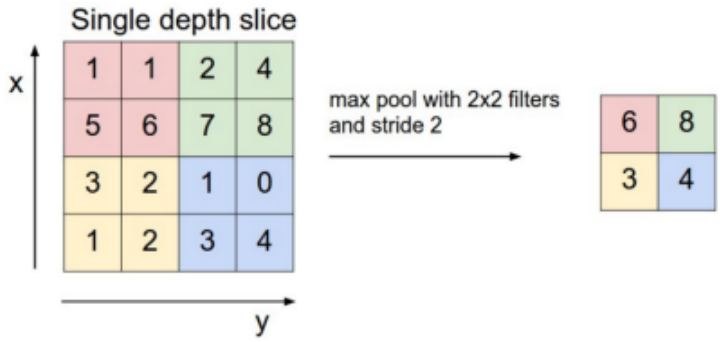


FIGURE 2.8: Example of max-pooling filter [12]

Pooling reduces the data size, leading to an improvement of the network efficiency. It is useful if the exact feature position is not relevant but rather whether a certain feature exists in the input at all [12].

2.3.8 Example of application field: Object Detection

With CNNs, a lot of progress has been made, not only with object recognition, but also with object detection. Multiple (meta)-architectures for object detection have been developed and iterated upon. The term meta-architecture is used to refer to object detection architectures that use similar approaches for detection.

Object detection deals with the localization and the identification of an object. Object localization and identification are two different tasks that are put together to achieve this singular goal of object detection. Object localization deals with specifying the location of an object in an image or a video stream, while object identification deals with assigning the object to a specific label, class, or description.

SSD MobileNet

Is an object detector released in 2017 as an efficient CNN architecture designed for mobile and embedded vision application. This architecture uses proven depth-wise separable convolutions to build lightweight deep neural networks.

The SSD architecture is a single convolution network that learns to predict bounding box locations and classify these locations in one pass. Hence, SSD can be trained end-to-end. The SSD network consists of base architecture (MobileNet in this case) followed by several convolution layers:

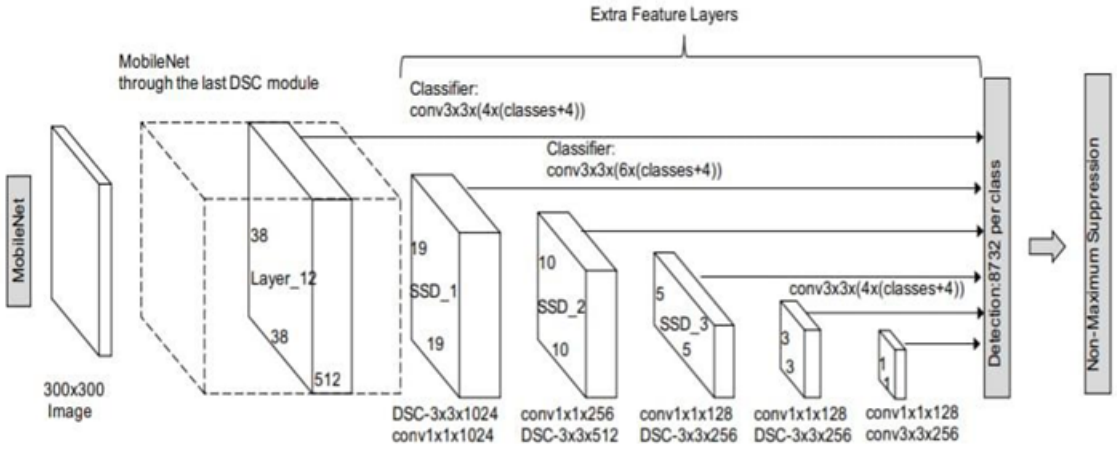


FIGURE 2.9: SSD MobileNet Architecture

By using SSD, we only need to take one single shot to detect multiple objects within the image, while regional proposal network (RPN) based approaches such as R-CNN series that need two shots, one for generating region proposals, one for detecting the object of each proposal. Thus, SSD is much faster compared with two-shot RPN-based approaches.

Evaluation Metrics

In this section, evaluation metrics used within object detection is covered. This includes the terms precision and recall, which are general within the field of machine learning, as well as metrics specific to object detection.

- **Precision and Recall**

To calculate the average precision of a model, precision and recall are used, defined in Equation 2.11 and 2.12. Precision measures how accurate the predictions are, measured in percentage of correct predictions. Recall measures how well the model finds all the positives. An overview of the terms true positive, false positive, false negative and true negative can be seen in Figure 2.10. As a hands on description of the terms with an example of the person class: true positive is when a person is detected as a person, false positive is when something else is detected as a person, false negative is

when the detector fails to recognize a person, true negative is when the detector does not detect something else as a person.

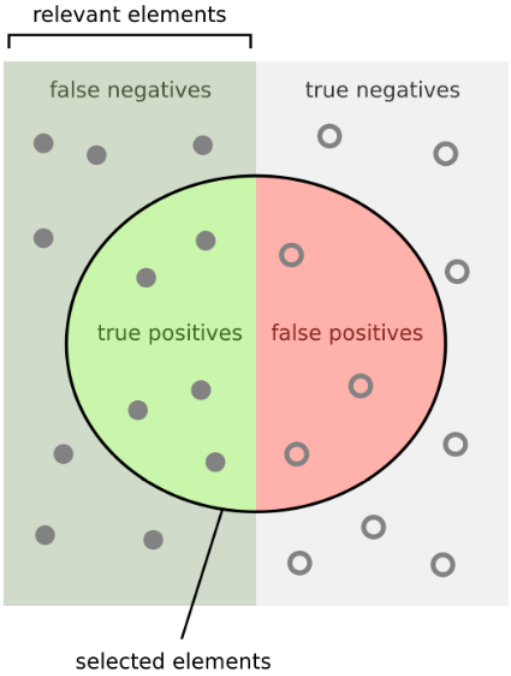


FIGURE 2.10: Precision and Recall

$$precision = \frac{true\ \ positive}{true\ \ positive + false\ \ positive} \tag{2.11}$$

$$recall = \frac{true\ \ positive}{true\ \ positive + false\ \ negative} \tag{2.12}$$

When the number of true positives, false negatives and false positives is determined from the test set, the precision and recall can be calculated. This can be plotted in a precision-recall curve. The Average Precision (AP_i) is defined as the area under the precision-recall curve for specific class (object) i and is given by Equation 2.13, where p is precision and r is recall.

$$AP_i = \int_0^1 p(r)\, dr \tag{2.13}$$

To evaluate a detector the metric mean Average Precision (mAP) can be used. MAP is the average value of the AP for N classes, it is defined by Equation 2.14.

$$mAP = \frac{1}{N} \sum_{i=1}^{N} AP_i \tag{2.14}$$

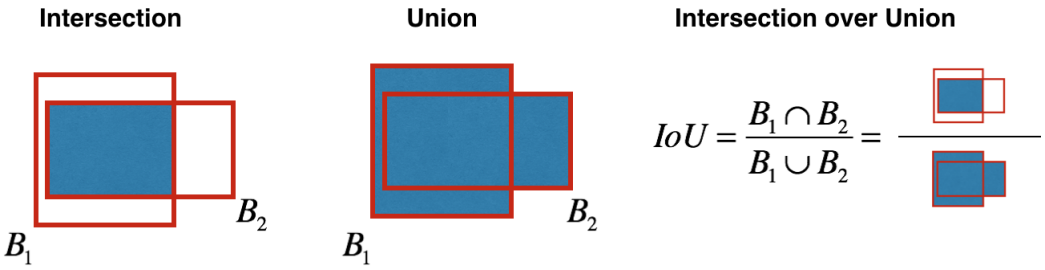


FIGURE 2.11: Intersection over union

- **Intersection over union**

To get an account of how well the detector locates the objects the intersection over union (IOU) is calculated, defined in Equation 2.15. IOU is the measure of the overlap of two bounding boxes. It measures how much the predicted bounding box overlaps with the ground truth, see Figure 2.11. A threshold value is set to label the detection as true positive or false positive. If the IOU is greater than the threshold it is considered as a true positive, and else a false positive.

$$IOU = \frac{area\ \ of\ \ overlap}{area\ \ of\ \ union} \tag{2.15}$$

- **Loss**

To evaluate the networks, three different losses were used: classification loss, localization loss and regularisation loss. These losses were provided by the Tensorflow Object Detection API and together form the total loss. The regularisation loss defined in section 2.3.3. The localization loss is a is a smooth L1 loss between the predicted bounding box correction and the true values.

$$\begin{aligned}
\mathcal{L}_{\text{loc}} &= \sum_{i,j} \sum_{m \in \{x,y,w,h\}} \mathbb{1}_{ij}^{\text{match}} L_1^{\text{smooth}}(d_m^i - t_m^j)^2 \\
L_1^{\text{smooth}}(x) &= \begin{cases} 0.5x^2 & \text{if } |x| < 1 \\ |x| - 0.5 & \text{otherwise} \end{cases} \\
t_x^j &= (g_x^j - p_x^i)/p_w^i \\
t_y^j &= (g_y^j - p_y^i)/p_h^i \\
t_w^j &= \log(g_w^j/p_w^i) \\
t_h^j &= \log(g_h^j/p_h^i)
\end{aligned} \tag{2.16}$$

where $\mathbb{1}_{ij}^{\text{match}}$ indicates whether the i-th bounding box with coordinates $(p_x^i, p_y^i, p_w^i, p_h^i)$ is matched to the j-th ground truth box with coordinates $(g_x^j, g_y^j, g_w^j, g_h^j)$ for any object. $d_m^i, m \in \{x, y, w, h\}$ are the predicted correction terms.The classification loss is a

softmax loss over multiple classes [31].

$$\mathcal{L}_{\text{cls}} = -\sum_{i \in \text{pos}} \mathbb{1}_{ij}^{k} \log(\hat{c}_i^k) - \sum_{i \in \text{neg}} \log(\hat{c}_i^0), \text{ where } \hat{c}_i^k = \text{softmax}(c_i^k) \tag{2.17}$$

where $\mathbb{1}_{ij}^{k}$ indicates whether the i-th bounding box and the j-th ground truth box are matched for an object in class k. pos is the set of matched bounding boxes (N items in total) and neg is the set of negative examples. SSD uses hard negative mining to select easily misclassified negative examples to construct this neg set: Once all the anchor boxes are sorted by objectiveness confidence score, the model picks the top candidates for training.

2.4 Reinforcement Learning

In Reinforcement Learning, an agent is supposed to learn a specific behavior by trial-and-error. The agent interacts with its environment to collect experiences. Figure 2.12 illustrates the basic concept behind Reinforcement Learning. At each time step t = 1, 2, 3, ... the agent is in a certain state $s_t \in$ S and takes one of the possible available actions $a_t \in$ A(s). The action changes the environment and the agent ends up in a new state a_{t+1}. Furthermore, it receives a reward R_{t+1} from the environment that serves as a feedback about how good it was to take action a_t in state s_t.

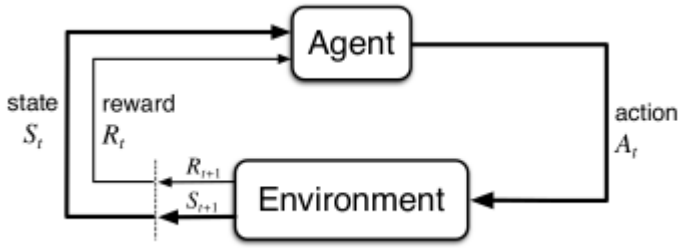


FIGURE 2.12: The agent-environment interaction in a Markov decision process [26]

This chapter gives an introduction to the basic concepts of classical Reinforcement Learning and serves as the foundation for the advanced Deep Reinforcement Learning approaches, discussed in section 2.5 [26].

2.4.1 Markov Decision Process

The Markov Assumption assumes an independence of past and future states, meaning that the state and the behavior of the environment at time step t are not influenced by the past agent-environment interactions a_1, ..., a_{t-1}. If the RL-task can fulfill the

Markov Assumption, it can be formulated as five-tuple Markov Decision Process (S, A, $P^a_{s,s\prime}$, $R^a_{s,s\prime}$, γ).

- Set of states S
- Set of actions A. A(s) is the set of available actions in state s.
- Transition probabilities $P^a_{s,s\prime} : (S \times A \times S) \longrightarrow [0, 1]$. It is the probability of the transition from s to $s\prime$ when taking action a in state s at time step t.
- Reward probabilities $R^a_{s,s\prime} : (S \times A \times S) \longrightarrow [0, 1]$ IR. It defines the immediate reward the agent receives after the transition from s to $s\prime$.
- Discount factor $\gamma \in [0, 1]$ for computing the discounted expected return.

The Markov Decision Process (MDP) is finite if the set of states S and actions A is finite.

2.4.2 Discounted Expected Reward

To train an effective agent, its goal should be to maximize the reward in the long run instead of just caring about the immediate return. Consider the environment of figure 2.13 with six different rooms. The agent starts in room one and the goal is to end up in room five. The immediate reward is the negative distance of the agent to room five. If the agent just cares about maximizing the immediate reward, it changes to room three because the immediate reward is higher than in room one or two. Unfortunately, it is not able to reach room five from there and it remains in room three. It will never reach the final goal. On the contrary, if the agent's effort is to maximize the expected return, it accepts to receive a lower immediate reward in room two, followed by higher immediate rewards in room four, six and five. The sum of immediate rewards is maximized.

The discounted expected return is the cumulative sum of possible future rewards and can be found in equation 2.18. The discount factor $\gamma \in [0, 1]$ rates the future rewards and defines how far in the future rewards are considered. If = 1 all rewards of the future are considered with the same weight. If $\gamma = 0$ just the immediate reward is taken into account.

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+2} + ... = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \tag{2.18}$$

It can be differentiated between episodic and continuous tasks.

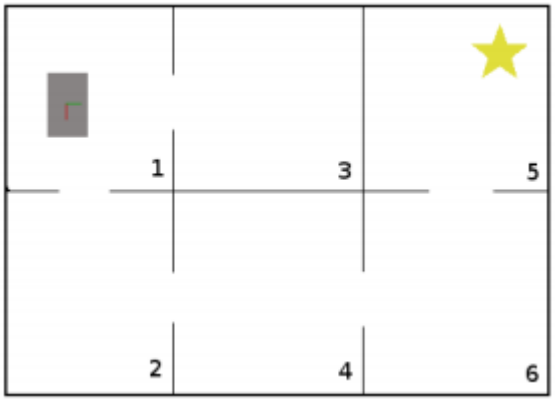


FIGURE 2.13: Example environment for RL

- Episodic: The training procedure can be divided into episodes. When the agent reaches a terminal state, the episode is over, the scene will be reset and the agent will restart in the next episode. The terminal state has an immediate reward of 0 and can be reached in T finite time steps.
- Continuous: The problem cannot be formulated in episodes and is a continuous ongoing problem so that T = ∞.

2.4.3 Policy and Value Functions

A policy π tells the agent how to behave. It models a probability distribution $\pi(\mathrm{a} \mid \mathrm{s})$ over the number of available actions a $\in$ A(s) for each state s, i.e. $\pi(A_t \mid S_t)$ is the probability of taking action (A_t in state S_t). The agent samples its next action from that probability distribution $\pi(\mathrm{a} \mid \mathrm{s})$. The value function $v_\pi(\mathrm{s})$ is an estimate of how good it is for the agent to be in state s. $v_\pi(\mathrm{s})$ is the expected discounted return of state s, if the agent behaves according to policy π (see equation 2.19).

$$v_\pi(s) = \mathbb{E}_\pi[G_t|S_t = s] = \mathbb{E}_\pi[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1}|S_t = s], \ \ for \ \ all \ \ s \in S \tag{2.19}$$

Above all, equation 2.19 can be formulated recursively. The recursive form is called Bellman Equation for v_π and is shown in equation D.5. In the Bellman equation, the value of state s is only dependent on the next possible states $s\prime$ while each state is weighted by the transition probability $P^a_{s,s\prime}$. Many Reinforcement Learning solutions approximate the optimal Bellman equation by approximating the value of the next states.

The action-value function q_π(s, a) is an estimate of how good it is to take action a in state s. q_π(s, a) is the expected return of taking action a in state s and thereafter behaving according to policy π.

$$q_\pi(s,a) = \mathbb{E}_\pi[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} | S_t = s, A_t = a] \quad for\ all\ s \in S\ and\ a \in A \tag{2.20}$$

Reinforcement Learning aims to find an optimal policy π^*. A policy is better than another policy $\pi \geq \pi\prime$ if the value function of the new policy is better $v_\pi(s) \geq v_{\pi\prime}(s)$ for all s $\in$ S. If the state-value function is optimal, an optimal policy was used by the agent. It is possible that there are multiple optimal policies, that lead to the same optimal state-value function. The optimal state-value function v^* can be defined as followed:

$$v^*(s) = \max_\pi v_\pi(s)\ \ for\ all\ s \in S \tag{2.21}$$

Furthermore, optimal policies result in the optimal action-value function q^*.

$$q^*(s,a) = \max_\pi q_\pi(s,a)\ \ for\ all\ s \in S\ and\ a \in A \tag{2.22}$$

Finally, the Bellman optimality equation in equation 2.23 can be derived from the previously introduced equations.

$$v^*(s) = \max_{a \in A(s)} q_{\pi^*}(s,a) \tag{2.23}$$

2.4.4 Temporal Difference Methods

The advantage of Temporal Difference (TD) methods is that they are applicable to online learning and continuous RL-tasks. The policy is updated in each time step t so that there is no need of completed episodes. Moreover, no model of the environment is required. Experiments showed that TD methods tend to converge more quickly than the Monte Carlo method.

In TD-methods, the true expected return G_t is not available and is approximated with the TD-target. The TD-target can also be seen as an approximation of the Bellman equation 2.23. The TD-target is determined by taking the sum of the immediate return and the discounted expected value of the next state: $R_{t+1} + \gamma V(S_{t+1})$. The pseudo-code of the general concept of TD-methods can be found in algorithm 1. In each iteration of each episode, an action A_t is retrieved from policy π for a given State S_t. The action A_t is executed and the agent is rewarded with R_{t+1} and transitioned to the next state S_{t+1}. Finally, the value-table V is updated with the TD-error, which

is the difference of the TD-target and V(S_t).

Q-Learning

Q-Learning is a popular off-policy TD control method, i.e. the policy π is not used for updating the Q-table. Only the maximum Q-value of the next state S_{t+1} is taken. The action-value update rule of Q-learning is shown in equation 2.24.

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)] \tag{2.24}$$

2.5 Deep Reinforcement Learning (DRL)

In the classical Reinforcement Learning, discussed in chapter 2.4, all approaches have a tabular setting, leading to essential disadvantages. On the one hand, the usage of a table limits the classical approaches to tasks with a low number of states and actions. In real world problems, the state space can quickly get large. It is not feasible to visit all possible states to retrieve the value for all action-state pairs. Furthermore, the size of the table is limited due to memory constraints in hardware. On the other hand, knowledge about similar states is not shared. This could lead to better representation and lower training times.

To overcome the mentioned restrictions, a common approach is to replace the value table with a Deep Neural Network as function approximator. Their ability to approximate non-linear functions and to extract relevant features from raw inputs makes it possible to generalize over unseen states.

2.5.1 Policy Gradient Methods

Policy Gradient Methods optimize the policy $\pi(\mathrm{a} \mid \mathrm{s}, \theta)$ directly instead of learning a value function and choosing the actions based on it (e.g ϵ-greedy policy). The quality of each policy can be measured by the policy's performance measure J(θ). The objective function of Policy Gradient Methods in equation 2.25 maximizes the scalar value J(θ).

$$\theta^* = \arg\max_\theta J(\theta) \tag{2.25}$$

The policy's parameter θ are updated via gradient ascent. Gradient ascent is the inverse of gradient descent and updates the parameters θ_t in the positive direction of the gradient of the policy's performance measure ∇_θ J(θ) (see equation 2.26). Furthermore, the learning rate α defines, how strongly one steps in the gradients direction.

$$\theta_{t+1} = \theta_t + \alpha \nabla_\theta J(\theta_t) \tag{2.26}$$

One advantage of Policy Gradient Methods is their stable convergence property because the policy is updated directly and thus improves smoothly at each time step. Valuebased methods update the value function at each time step. A small change in the value function can lead to a drastic change in the policy output. Hence, value-based methods often deal with big oscillations during training. Especially, Policy Gradient Methods can deal with infinite and continuous action spaces. Instead of determining a Q-value for each possible discrete action, the action can be estimated directly, e.g. the speed of the mobile robot is estimated directly by the agent. The third advantage of Policy Gradient Methods is their ability to learn stochastic policies, i.e., actions are chosen with a certain probability. It is especially necessary for uncertain, partially observable environments. The big disadvantage of Policy Gradient Methods is that they rather converge to a local maximum than to the global optimum [26].

Actor-Critic Architecture

A Policy Gradient Method that makes use of the value function v(s) to learn the policy parameters θ is called Actor-Critic Architecture. Figure 2.14 illustrates the basic idea of the architecture: The Actor represents the current policy and generates an action a for a given input state s. The Critic represents the value function v(s) and computes the expected value for a given input state. A common practice is to update both networks with the TD-Error, discussed in chapter 2.4.4. The expected values of the current and the next state that contribute to the TD-Error are estimated with the Critic. Thus the Critic's output contributes to the Actor's update essentially.

Proximal Policy Optimization (PPO)

PPO [10] is a popular state-of-the-art Policy Gradient Method. It is supposed to learn relatively quickly and stable while being much simpler to tune, compared to other stateof-the-art approaches like TRPO, DDPG or A3C . This makes PPO often the first choice when it comes to solving a problem for the first time. PPO strongly builds on Trust Region Policy Optimization (TRPO). It applies the key concepts of TRPO like Importance Sampling, that provides better data efficiency as well as an extended version of TRPO's KL penalty, that controls the update size in the optimization step. Moreover, PPO presents an alternative, simpler method called Clipped Surrogate Objective for controlling the optimization step size.

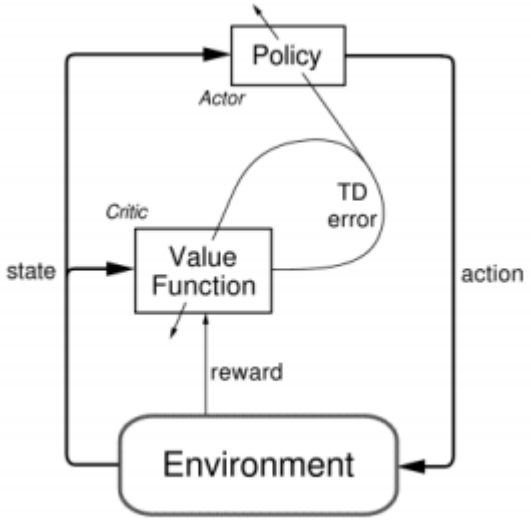


FIGURE 2.14: Actor-Critic Architecture [26]

- **Importance Sampling** In the REINFORCE algorithm (see Appendix D), at each time step a new trajectory is generated, the policy learns from it and the trajectory is thrown away. To achieve better data efficiency, importance sampling is applied in PPO. Trajectories that has been collected with older policies are reused in newer updated policies. Importance Sampling estimates the expected value of f(x) for distribution p (new policy) by sampling from q (old policy), i.e. X_i has been sampled from the data distribution q. Applying Importance Sampling leads to a new objective function – called surrogate function L(θ).

- **Adaptive KL Penalty** To ensure stable updates, the step size in the optimization step can be controlled with Trust Region from. It prevents the optimization from taking too big steps and from overshooting the maximum. In each step, the difference between the updated π_θ and the old policy π_{θ_old} is measured through the KL Divergence.

  The KL Divergence is added to the surrogate function L(θ) as penalty, multiplied by a factor β. If the difference between the old and new policy is big, the objective function is punished strongly.

  Difficulties have been experienced with finding a good constant β, that performs well over the whole training process. That is why an adaptive β is applied in PPO. If the KL divergence d is very small, β gets smaller as well and if it is high, β gets bigger. Still the initial and the target distance d_{trag} have to be set.

- **Clipped Surrogate Objective** PPO presents a second method to control stable updates by restricting the update size from one policy to another. Instead of

adaptive KL penalty, it applies a much simpler and more effective way to realize the restriction by introducing the clipped surrogate function. It clips the probability ratio $r_t(\theta) = \frac{\pi_\theta(S_t,A_t)}{\pi_{\theta_old}(S_t,A_t)}$ between the range of $[1-\epsilon, 1+\epsilon]$.

Deep Deterministic Policy Gradient (DDPG)

Deep Deterministic Policy Gradient (DDPG) is based on the Deterministic Policy Gradient Algorithms from and uses an Actor-Critic Architecture. Furthermore, it adapts concepts from DQN (see Appendix D) to stabilize learning, but addresses continuous action space at the same time and is, therefore, more applicable for controlling tasks.

It makes use of an experienced replay buffer to provide independent and uncorrelated data to the Deep Neural Networks. In addition, a variant of the frozen target network mechanism has been integrated, called soft target update. Instead of copying the weights to the target, the target networks (both Actor and Critic) are updated continuously according to equation 2.27 and slowly approach the original parameters. s defines the tracking speed, i.e., the smaller τ, the slower the target weights approach the original weights.

$$\theta^- \leftarrow \tau_s\theta + (1-\tau_s)\theta^- \tag{2.27}$$

DDPG is built on an Actor-Critic architecture with Deep Neural Networks. The Actor $\mu(s|\theta^\mu)$, as well as the Critic $Q(s,a|\theta^Q)$, have both a duplicate target network with older weights to stabilize learning. They are denoted with a hyphen: $\mu^-(s|\theta^\mu)$ and $Q^-(s,a|\theta^Q)$. Both networks are trained by forwarding uniformally sampled experiences from the replay buffer (S_t, A_t, R_{t+1}, S_{t+1}). The Critic is updated by back-propagating the gradient of the squared TD-error from Q-learning.

The Actor is updated with the Deterministic Policy Gradient by applying gradient ascent to the output of the Critic with respect to the state s and the estimated action by the actor $\mu(S_t|\theta_i^\mu)$

In order to apply exploration to continuous action space, noise N is added to the actor's output $\mu(s|\theta^\mu)$. The noise is generated by the Ornstein-Uhlenbeck process.

Soft Actor-Critic (SAC)

Soft Actor-Critic (SAC) [27] is an algorithm that optimizes a stochastic policy in an off-policy way, forming a bridge between stochastic policy optimization and DDPG-style approaches. It incorporates the clipped double-Q trick, and due to the inherent stochasticity of the policy in SAC, it also winds up benefiting from something like target policy smoothing.

A central feature of SAC is entropy regularization. The policy is trained to maximize

a trade-off between expected return and entropy, a measure of randomness in the policy. This has a close connection to the exploration-exploitation trade-off: increasing entropy results in more exploration, which can accelerate learning later on. It can also prevent the policy from prematurely converging to a bad local optimum.
To explain Soft Actor Critic, we first have to introduce the entropy-regularized reinforcement learning setting. In entropy-regularized RL, there are slightly-different equations for value functions.

Entropy is a quantity which, roughly speaking, says how random a random variable is. If a coin is weighted so that it almost always comes up heads, it has low entropy; if it's evenly weighted and has a half chance of either outcome, it has high entropy.

2.6 Conclusion

In this chapter, some basic notions necessary for the realization of the project defined. We presented the concepts of image processing, deep learning, machine learning and deep reinforcement learning as well as the measures that will be used to answer the problem of this project. In the next chapter, we will present the architecture and the working environment to accomplish the proposed solution.

Chapter 3

Architecture and Working Environment

3.1 Introduction

The theoretical studies carried out in the preceding chapter will be combined to draw up an architecture which meets the functional requirements of the solution. This chapter is therefore devoted to the architecture of the project, working environment and simulation environment.

3.2 Working Environment

In this section, the working environment is briefly presented by displaying the hardware and software tools and the simulation environment used in order to implement and validate the solution.

3.2.1 Hardware Tools

SunFounder Smart Video Car Kit V2.0(PiCar-V)

Picar-V is an open-source robot learning kit based on Raspberry Pi as shown in figure 3.1 and equipped with :

- **Raspberry Pi** (Raspberry Pi 4): This is a low-cost computer with a processing speed of 1.5 GHz and a memory of 4 GB. It is equipped with a customized version of the Linux system, supports Bluetooth, WIFI communication, and has rich support for i2c, etc. The agreement amount is GPIO (General Purpose Input/Output) port, which is the calculation brain for our auto-driving car. It is located under the robot HATS.
- **Wide-angle USB Webcam** : The resolution is 640 x 480 and the viewing angle is 120 degrees. It is our only environmental sensing device, which is our eyes.

- **PCA9685 PWM driver** [22] : Includes an i2 C-controlled PWM driver with a built-in clock to drive the modified servo system.
- **DC Gear Motor** [18] : A Direct Current (DC) motor is a rotating electrical device that converts direct current, of electrical energy, into mechanical energy.
- **DC motor driver TB6612** [25] : is a device that can coordinate in a predetermined manner the performance of an electric motor. It include a manual or automatic means for starting and stopping the motor, selecting forward or reverse rotation, selecting and regulating the speed, regulating or limiting the torque, and protecting against overloads and electrical faults.
- **SF006C Servo** [23]: is a rotary actuator that allows for precise control of angular position, velocity and acceleration. It consists of a suitable motor coupled to a sensor for position feedback. It also requires a relatively sophisticated controller, often a dedicated module designed specifically for use with servomotors.

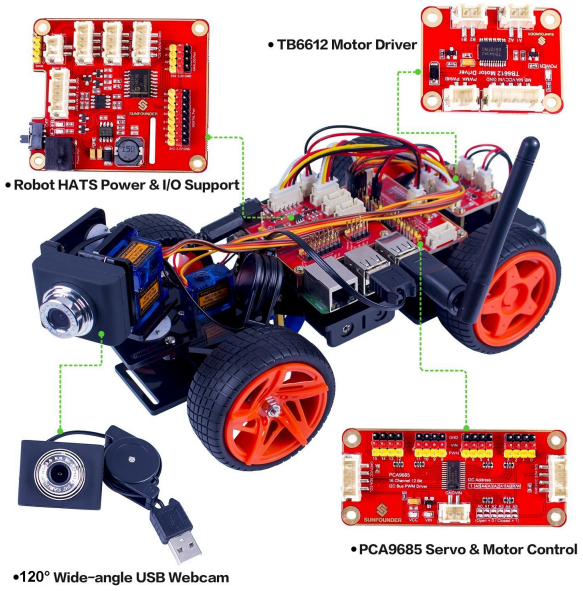


FIGURE 3.1: PiCar-V

Artificial intelligence models and computer vision techniques usually require large computing and memory capacities and powerful graphics processors for them to function. But sending the data to the server for processing and then retrieving the results generally introduces latency. Consequently, for applications which require real-time processing, it is better to analyze the data as soon as they are acquired, this can be done with on-board systems. That's why the Intel Neural Compute Stick 2 is required.

Intel Neural Compute Stick 2

The intel NCS2 (Figure 3.2) is a tiny fanless deep learning device used to learn AI programming at the edge. NCS2 is powered by the same low power high-performance VPU that can be found in millions of smart security cameras, gesture-controlled drones, industrial machine vision equipment, and more. The Intel Neural Compute Stick 2 enables rapid prototyping, validation and deployment of DNN inference applications at the edge. Its low-power VPU architecture enables an entirely new segment of AI applications that aren't reliant on a connection to the cloud. The NCS2 combined with Movidius^{TM} Neural Compute SDK [3] allows deep learning developers to profile, tune, and deploy CNN on low-power applications that require real-time inferencing. The NCS2 delivers 8X performance boost over the previous generation.



FIGURE 3.2: Intel Neural Compute Stick 2

It is right that NCS2 allow us to deploy deep learning architecture but only for testing because the training phase needs so mush power and calculation that goes beyond its capabilities. In another way, to implement a model in an embedded system a development phase is needed, so powerful server needed with computers that can help us with a development and distance access to this embedded system.

Development Servers

The materials that have been used to develop, train and validate the solution feature are a computer that belong to SAGEMCOM, personal laptop and a hosted Jupyter notebook service Colaboratory which is a product of Google Research that offer free access to its IT resources [19].
The host company computer specifications :

- CPU : Intel(R) Core (TM) i5-8500
- RAM : 16 GB
- GPU : GeForce GT 730

Laptop

- CPU : Intel(R) Core (TM) i7-4510
- RAM : 8 GB
- GPU : GeForce 820M

Colaboratory specifications:

- CPU : Intel(R) Xeon(R)
- RAM : 12 GB
- GPU : Nvidia K80 / T4 / P4 / P100

3.2.2 Software Tools

To develop an autonomous driving solution, a wide range of software and technologies are used.

Integrated Development Environment :

- **Visual Studio Code [15] :** Is a source-code editor developed by Microsoft for Windows, Linux and macOS. Language extensions can be found and downloaded for free from the VS Code Marketplace.
- **Unity :** Unity is a cross-platform game engine developed by Unity Technologies.

Libraries :

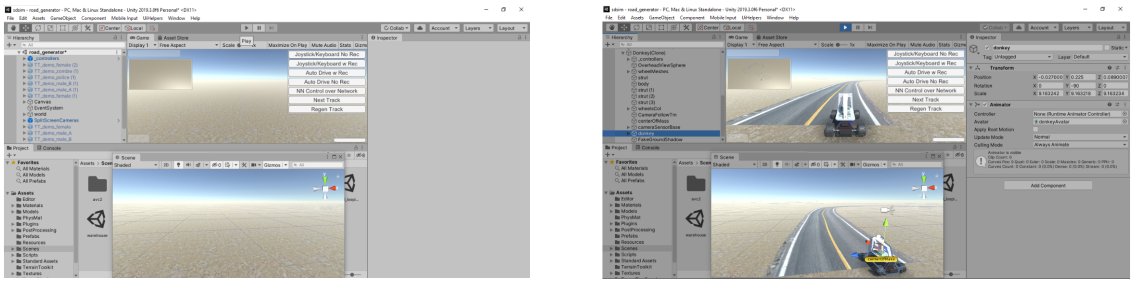
- **Donkey Car [20] :** Donkey is an open source project and a high level self driving library written in Python. It was developed with a focus on enabling fast experimentation and easy contribution.
- **gym-donkeycar [11] :** OpenAI Gym Environments for Donkey Car.
- **OpenCV :** is a library of programming functions mainly aimed at real-time computer vision. The library is cross-platform and free open-source.
- **OpenVINO :** is a comprehensive toolkit for quickly developing applications and solutions that emulate human vision. Based on Convolutional Neural Networks, the toolkit extends computer vision workloads across Intel hardware, maximizing performance.

- **Pandas [2] :**It is easy to use for manipulating and analyzing data. In particular, it proposes data structures and operations for manipulating numeric arrays and time series.
- **PyTorch [5] :**An open source machine learning framework that accelerates the path from research prototyping to production deployment.
- **SunFounder-PiCar [24] :** python API for PiCar-V developed by SunFounder.
- **TensorFlow [9] :** TensorFlow is an end-to-end open source platform for machine learning. It has a comprehensive, flexible ecosystem of tools, libraries and community resources that lets researchers push the state-of-the-art in ML and developers easily build and deploy ML powered applications.
- **TensorFlow Object Detection API** : open source framework built on top of TensorFlow that simplifies the process of training models to detect objects by using pre-trained models.

3.3 Simulation Environment

This section introduces the used simulation environment to train our RL agent which is the second approach. The modelling of all objects as well as the robot is presented in detail. Eventually, just image data coming from a front camera is considered as input to train the model with.
The first step is to create a high fidelity simulator for PiCar-V. Fortunately, someone from the Donkey Car [20] community has generously created a Donkey Car simulator in Unity. However, it is specifically designed to perform behavioural learning (i.e. save the camera images with the corresponding steering angles and throttle values in a file for supervised learning), but not cater for reinforcement learning at all. Hopefully, the gym-donkeycar is here and made it compatible with reinforcement learning so we can manipulate the simulated environment through calling reset() to reset the environment, step(action) to step through the environment, etc.
The Unity simulator also comes with python code for communicating with Unity. The communication is done through the Websocket protocol. Websocket protocol, unlike HTTP allows two-way bidirectional communication between server and client. In our case, our python “server” can push messages directly to Unity (e.g. steering and throttle actions), and our Unity “client” can also push information (e.g. states and rewards) back to the python server.
The pre-bulid simulation as shown in Figure 3.3 is composed of 2 modes: Construction mode and play mode. In the construction mode, we can save, build, import,

modify, etc. The play mode is the visualisation mode for our constructed simulation where it can show us the same simulation showed after the build step. Of course, we can do some manipulation in this mode and notice results in real-time.



(A) Construction Mode (B) Play Mode

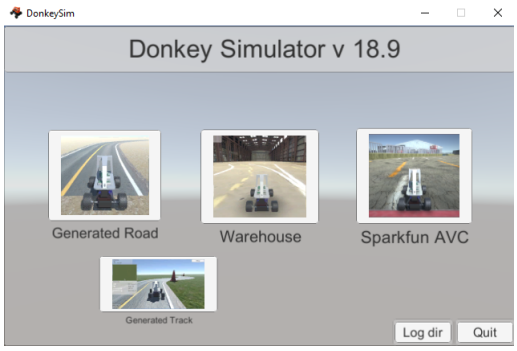
FIGURE 3.3: Construction and Play Mode

The simulation environment is composed of 4 environments which are: "donkey-warehouse-v0", "donkey-generated-roads-v0", "donkey-avc-sparkfun-v0" and "donkey-generated-track-v0" (Figure 3.4a). The difference between them is only the type of road so we can choose any one of them which is, for example, the first one (Generated Road). As shown in Figure 3.4b there is a lot of buttons; each button represents an option. For example, the first one (Joystick Keyboard No Rec) is for the navigation mode in which we can control the car (Figure 3.4d) and the last one (Regen Track) is responsible for track changes (Figure 3.4c).

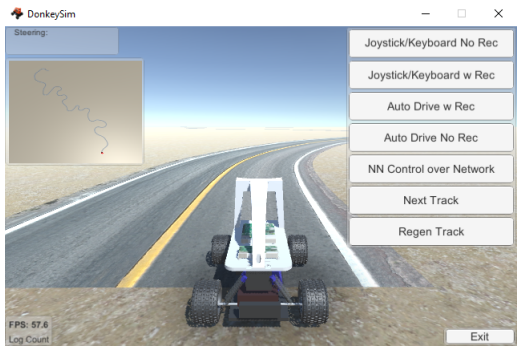
In this project, not only lane detection needed to satisfy our purpose but also Traffic signs and person detection is necessary for this aim. So the Donkey simulator needs a rework because it consists of only road track.
Figure 3.5a shows us the different updates applied at this simulator. We will focus at first on the person object because traffic signs need some specific construction in the simulation so the RL-agent can train successfully with these signs. These lego persons are available for free in the asset store of Unity. So we put them randomly on the track in such a way they are placed close to one of the lines so the robot car can see this latter and move accordingly between the other line and the person (Figure 3.5a). We updated also the the track for more experiments.

The most important data that unity simulator offers us about our agent are its coordinates and physical interaction with other objects. With these data in hand, we can figure out if the car is outside the track or not because we have line lanes middle coordinates, and also we can find if the robot hit something or not. For the hit information, we need a "box collider" as represented in Figure 3.5b related to the object that we wish to avoid so if the robot hit something a collision occurred and information from this collision is returned back. So these gathered pieces of information are necessary for our RL algorithm to know the end of each training episode. Therefore

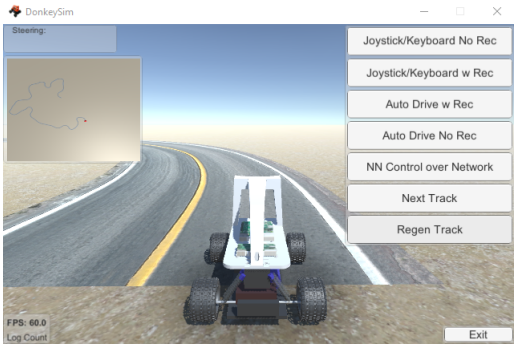
map between error(end of an episode) and images is considered for learning, so after the training completes, the agent can move accordingly to the camera vision without any mistake. It is right that our simulator gives us also images captured by the robot, but we don't need them for advanced processing to gather more information about the outside world otherwise Reinforcement Learning will be not a self-learning method.



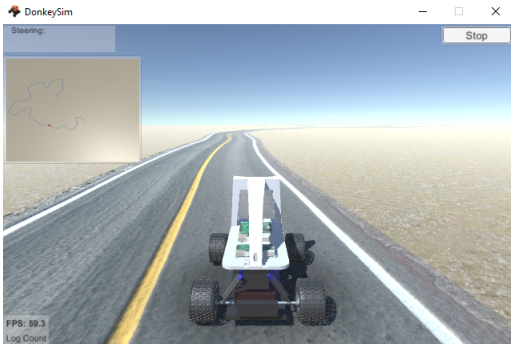
(A) Donkey Simulator first interface



(B) First Generated Track

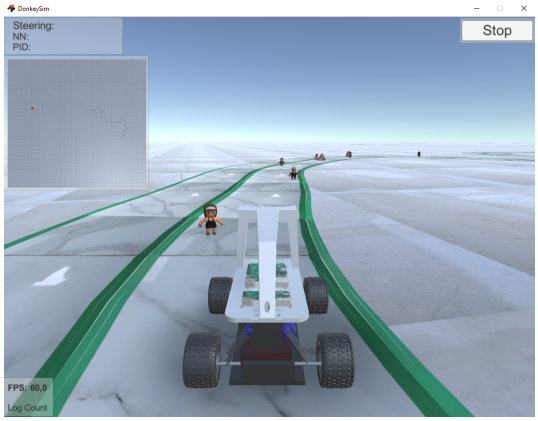


(C) Regenerated Track



(D) Control Mode

FIGURE 3.4: Donkey Simulator



(A) Donkey simulator updated



(B) Person Collider

FIGURE 3.5: New Updated Donkey Simulator

3.4 Solution Architecture

The theoretical studies carried out in the previous chapter will be present together to establish an architecture that meets the functional requirements of the solution. This section is, therefore, devoted to the presentation of the overall structure of the project and the development of the different used approaches.

3.4.1 Global Architecture

As shown in the figure below, images captured by the webcam are sent at first to the Raspberry Pi, then depending on which approach we use, they sent to the NCS2 for more processing. For the first approach, every frame sent is followed by an extra copy so the two developed algorithms can process simultaneously and independently and then an action is taken (steering, throttle) based on these two. Then for the second approach, we use only Raspberry Pi where images sent for a first algorithm, so next, the output of this latter transferred for processing in the second algorithm (Figure 3.6).

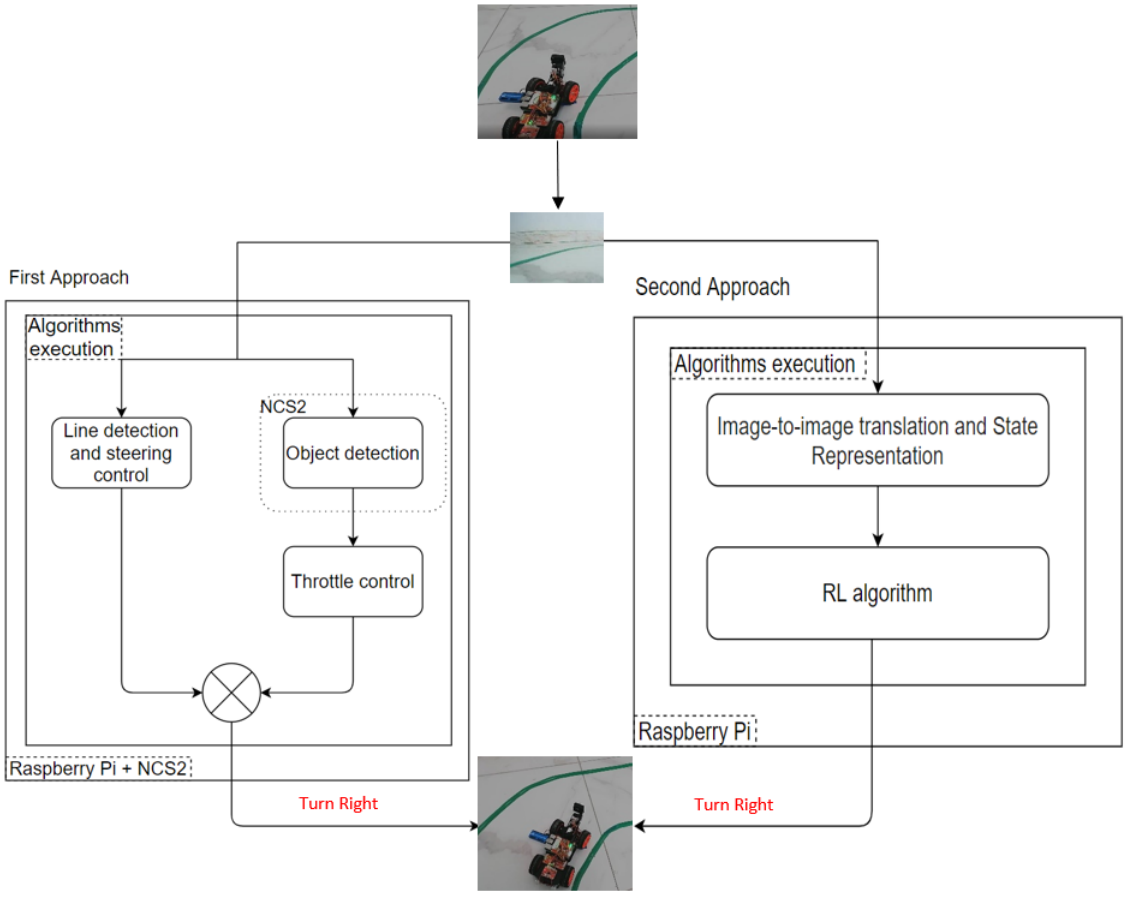


FIGURE 3.6: Global Architecture

3.4.2 First Approach

The objective of this section is to detect lines, pedestrians and traffic signs using image processing and deep learning techniques. So at first, we will use image processing for lane detection then deep learning for traffic sign and pedestrian detection.

Lane Detection and Steering Control Method

The objective of this section is to detect lines and act accordingly to these latter so the robot can finally navigate autonomously between lines. Figure 3.7 show us essential steps for developing this method.



FIGURE 3.7: Lane Detection and Steering Control Architecture

- Lane detection

In this step, we developed a hand-tuned code to detect colour, edges and line segments using Canny Edge Detection and Hough Line Transform. The process of this development needs validation of performance at the same time so we can identify where is the problem if it occurs.

- Steering Control and Evaluation

After we get a good lane lines detector, we need to steer the car so that it will stay within lines. So, this step is for guessing which line segments belong to the left or right lane line and manipulating the steering angle to achieve a stable drive.

Traffic Sign and Pedestrian Detection

The objective of this section is to detect Traffic Signs and Pedestrians using Transfer Learning to Adapt a Pretrained MobileNet SSD Deep Learning Model. Since it exists many objects : "Stop", "Construction", "Person", etc. So, this is a multi-class problem. Figure 3.8 shows us the different steps for developing this solution.

- Data Collection

The purpose of this step is to find and accurately determine the data to be analyzed, define their quality, and discover the distribution of this data between different classes. As the implementation of this data requires time and effort considerable, we recovered a tagged database available on the WEB.

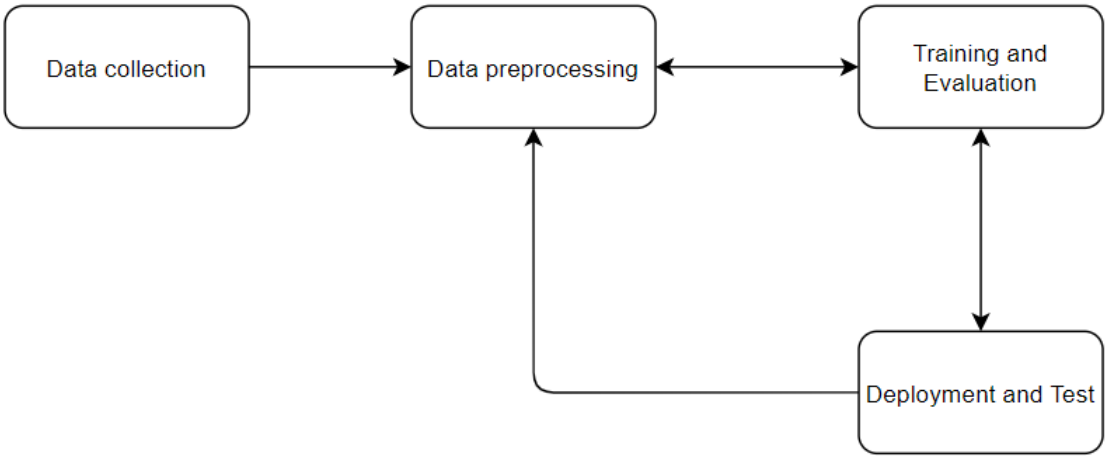


FIGURE 3.8: Steps for developing object detection method

- Data Preprocessing

It is a phase of cleaning and reorganizing the data according to the needs of the project. It also includes the recording of the data to make them compatible with the algorithm used. Manual treatments required for this kind of prior proceedings.

- Training and evaluation

In the training step, it is often to try different deep learning architectures, so in the final, we can choose the one with the best performance. Unfortunately, our NCS2 allow us only to use one specific algorithm for object detection problem which is SSD Mobilenet. So, this step includes setting up and training this model with the transfer learning method and turning back to the data preprocessing phase if it needed.

- Deployment and Test

This phase consists of calculating the performance of the model with the Mean Average Precision (map) and testing in real-time with our robot. So we can improve the result by going back to the step of data preprocessing.

Throttle Control

After the previous step realised successfully, we need to tell the car exactly what to do when it encounters each object. So this step is just for making rules. For example, tell the car to stop if it sees a pedestrian, or drive slower if it sees a lower speed limit sign, etc. Figure 3.9 show us exactly what rules applied for each specific detected object in each frame.

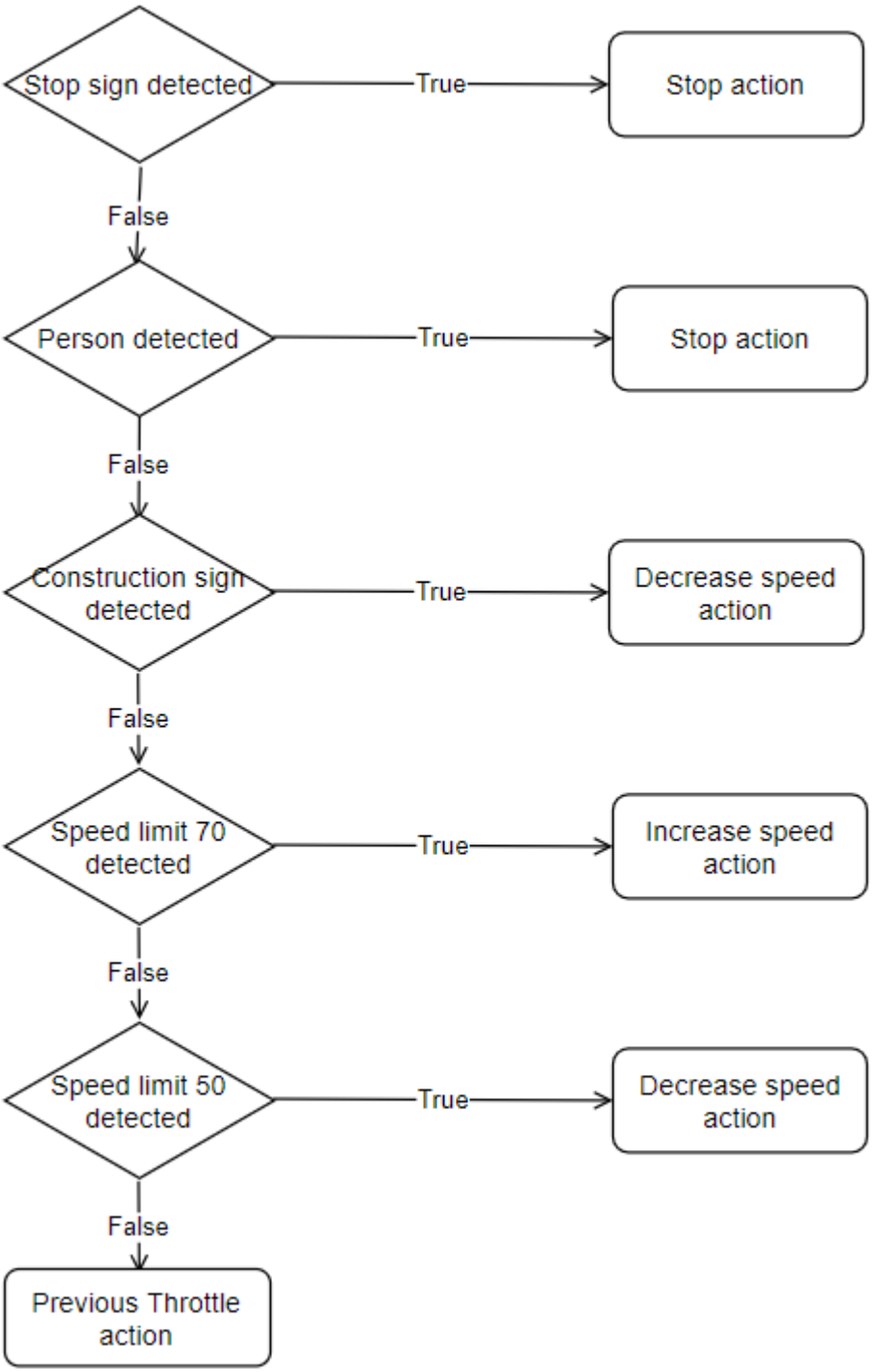


FIGURE 3.9: Actions applied for each specific detected object

3.4.3 Second Approach

The objective of this section is to detect lines and Pedestrians using Deep Reinforcement Learning. This time traffic signs not included because The strict time frame of the project did not allow us since it needs more training and complicated development to minimize the gap between the simulation and real environment, so we have experienced only with lines and pedestrian. The figure shows us the different steps for developing this solution.
As shown in Figure 3.10 there are two algorithms which the output of the first is the input of the other. We decided to put these latter in one section together because they are dependent on each other.

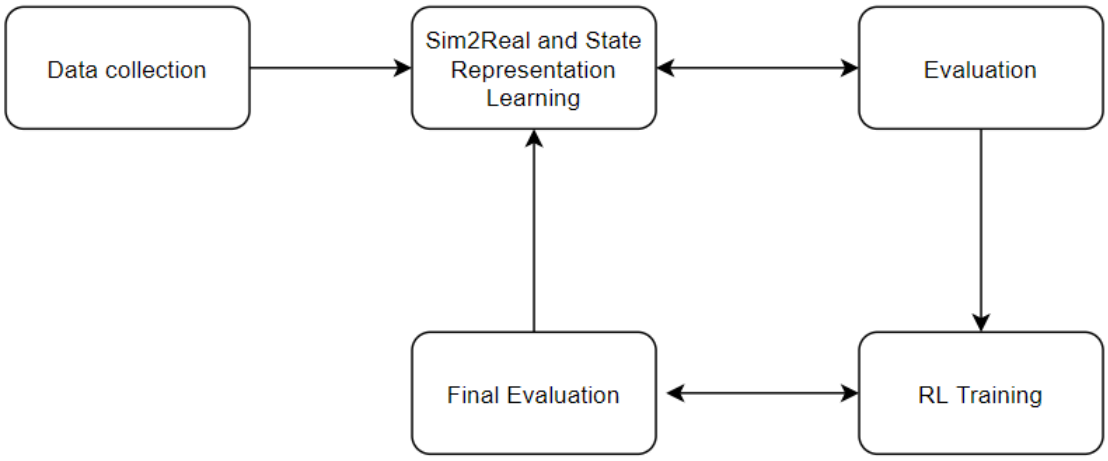


FIGURE 3.10: Steps for developing the second approach

Data collection

The purpose of this step is to collect the necessary data for the next step. The data is images gathered from the simulator robot by its front camera and also from the real one by its Webcam.

Sim2Real and State Representation Learning

In this step, we train a feature extractor to compress the image to a lower-dimensional space. The model is trained to reconstruct the input image but contains a bottleneck that forces it to compress the information. This step of extracting relevant information from raw data is called State Representation Learning. That notably allows to reduce the search space and therefore accelerate training.
In addition, we need later the transition from the simulation environment to the real world. Due to the heavy training times and safety issues, it is not possible to train

robots in the real world. That's why we trained also an image to image translator so we can minimize the gap between simulation and reality.

State representation Evaluation

The most accurate metric to evaluate our state representation performance is through the evaluation of the RL algorithm result, that means after training these two models. So, this process takes a while, especially if training goes wrong for the previous step then all the next process will be useless. The solution is to evaluate manually without losing so much time, and that with just visualising some regenerated images from the test data. This process helps us to manage the time with directly returning to the SR training phase if some regenerated pictures looked impossible to distinguish from the real one.

RL Evaluation

This step is essential for choosing the best RL algorithm that matches our need. This is done by testing at first the different performances of algorithms in the simulation then if we find more then one algorithm doing well with approximate results, so we will translate to test the performance in the real world. If we find just one algorithm doing well, then this is the best choice for us for real-world test performance.

3.5 Conclusion

This chapter first presented a detailed description of the different tools and environments used to accomplish the project. Then an abstraction of the process of this latter is described by a simple architecture.

Chapter 4

Implementation and Results

4.1 Introduction

In the previous chapters, we presented the necessary concepts for a deep understanding of the project and the detailed architecture of the solution by describing two adopted approaches. This last chapter aims to describe the steps of realization of these two approaches and give a final evaluation to conclude which is the best one.

4.2 First Approach

4.2.1 Lane Detection and Steering Control Method

Autonomous Lane Navigation system has two components, lane detection and steering control. Lane detection's job is to turn a video of the road into the coordinates of the detected lane lines. Once we detected lines in a single image, we can detect lane lines in a video by simply repeating the same steps for all frames in a video.

Lane detection

The original image captured by the Webcam of the robot is used all along this section as shown in the figure below.



FIGURE 4.1: Original image

First we isolated all the green areas on the image by turning the colour space, which is RGB (Red Green Blue) into the HSV (Hue Saturation Value) colour space (Figure 4.2).

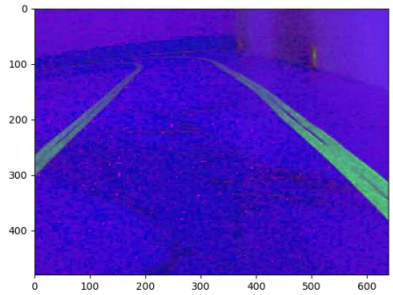


FIGURE 4.2: Image in HSV colour

Once the image is in HSV, we extracted all the greenish colours from the image to become a mask one as shown in Figure 4.3. This is by specifying a range of the colour Green which is 60-180

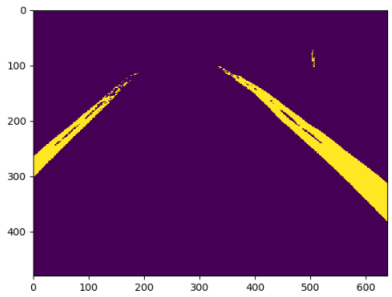


FIGURE 4.3: Green Area Mask

Next, we detected edges in the green mask so that we can have a few distinct lines that represent the green lane lines (Figure 4.4). For this we used Canny Edge Detection.

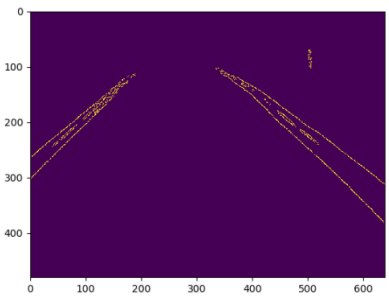


FIGURE 4.4: Edges of All Green Areas

From Figure 4.4, we see that we detected quite a few green areas that are not our lane lines. A closer look reveals that they are all at the top half of the screen. So we

simply crop out the top half because we only care about detecting lane lines that are closer to the car (Figure 4.5).

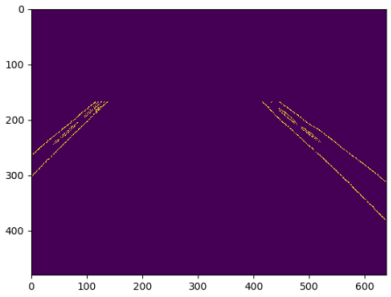


FIGURE 4.5: Cropped Edges

In the cropped edges image above, we found four long lines, which represent two lane lines. So, we extracted the coordinates of these lane lines from the yellow pixels by applying the Hough Line Transform (chapter 2.2.3) which gives as the illustrated figure 4.6 obtained by displaying the received coordinates.

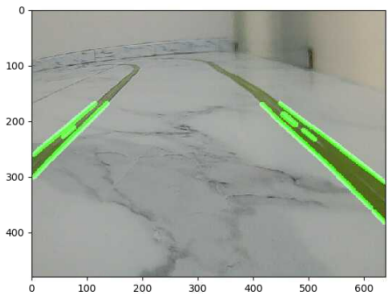


FIGURE 4.6: Lines segments

Now that we have many small line segments with their endpoint coordinates. We classified these line segments into two groups, by taking the average of the slopes and intercepts of the line segments to get the slopes and intercepts of left and right lane lines. So, figure 4.7 represent the combination of line segments into two lines. Also There are a couple of special cases.

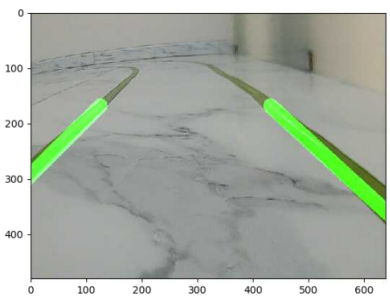


FIGURE 4.7: Combination of Line Segments into Two Lines

- One lane line in the image: In normal scenarios, we would expect the camera to see both lane lines. However, there are times when the car starts to wander out of the lane, at this time, the camera may only capture one lane line. That's why we checked if there are one or two lines detected so we can classify into one or two groups.

- Vertical line segments: Vertical line segments are detected occasionally as the car is turning. Although they are not erroneous detection, because vertical lines have a slope of infinity, we can't average them with the slopes of other line segments. For simplicity's sake, we chose to just to ignore them. As vertical lines are not very common since the Webcam is generally pointing at the same direction as the lane lines, doing so does not affect the overall performance of the lane detection algorithm.

Setting parameters is really a trial and error process. The Appendix B.1 show us the values that worked well for our robot car running between solid green lane lines.

Steering Control

In this scenario, we computed the heading direction by simply averaging the far endpoints of both lines. The red line shown in the figure 4.8 is the heading.

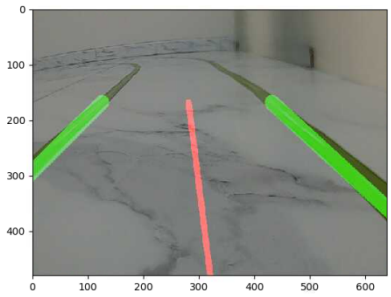


FIGURE 4.8: Heading direction

If we only detected one lane line, this means that we need to steer hard towards the same direction as the detected line, so we can continue to follow the lane. So, we set the heading line to be the same slope as the only detected line (figure 4.9).

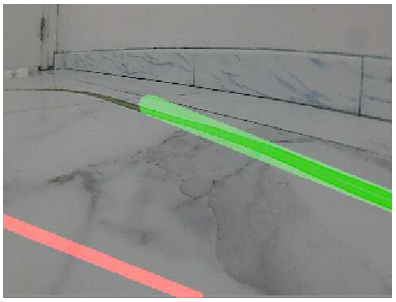


FIGURE 4.9: Heading direction for one line detected

Initially, when we computed the steering angle from each video frame, we simply told the PiCar to steer at this angle. However, during actual road testing, we have found that the PiCar sometimes bounces left and right between the lane lines, sometimes go completely out of the lane. We then found out that it is caused by the steering angles, computed from one video frame to the next frame, are not very stable. Some times, the steering angle may be around 90 degrees (heading straight) for a while, but, for whatever reason, the computed steering angle could suddenly jump wildly, to say 120 (sharp right) or 70 degrees(sharp left). As a result, the car would jerk left and right within the lane. So our strategy to stable steering angle is the following: if the new angle is more than 5 degree from the current angle, just steer up to 5 degree in the direction of the new angle.

For lane detection, evaluation results are discussed in section 4.4.

4.2.2 Pedestrian and Traffic Sign Detection

The Traffic sign and pedestrian detection model was defined by various steps, as indicated in section 3.4.2 of the previous chapter. In this section we will use Transfer Learning to Adapt a Pretrained MobileNet SSD Deep Learning Model to Detect Traffic Signs and Pedestrians with NCS2.

Data Collection

The images of the traffic signs used in this first approach are gathered from the "BelgiumTS Dataset", provided by the public research university "ETH Zurich" [28]. This database is made up of 7272 images which are all JP2 (JPEG 2000) type and 1628×1236 size with 210 classes and 13480 annotations (detected object coordinates and its class id). Figure 4.10 show us examples from this data.

For annotations, a TXT file is available, containing for each detected object its coordinates, name of the corresponding image and the class id to which it belongs. Class Ids for this data are provided in a separate folder. Figure 4.11 show us different types of traffic signs with their corresponding Class Ids which mentioned as the name of each sub-figure.

Pedestrians data-set (Figure 4.12) is gathered from the "LEGO Minifigures" competition hosted in Kaggle [32]. This data is made up of 352 images which are all 512×512 size, and it's made for classification of different categories only, so no annotations of detected boxes are available.



FIGURE 4.10: Traffic Signs Data Examples



(A) A31 (B) B5 (C) C35 (D) C45 (E) D5 (F) D9 (G) E1 (H) E3 (I) E9a

FIGURE 4.11: Traffic Signs different class types Examples

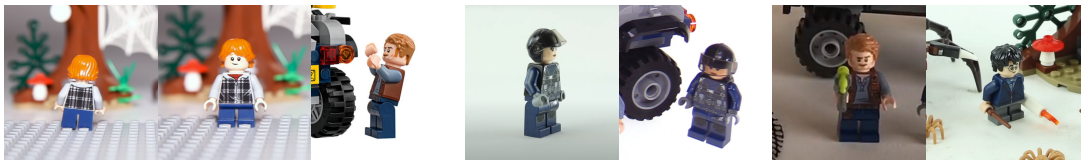


FIGURE 4.12: Pedestrians Lego Data Examples

Data Preprocessing

In this project, we will use just four different traffic signs because we can't use too much classes for testing our robot performance and the number of images of these four is the highest all over the most known traffic signs.

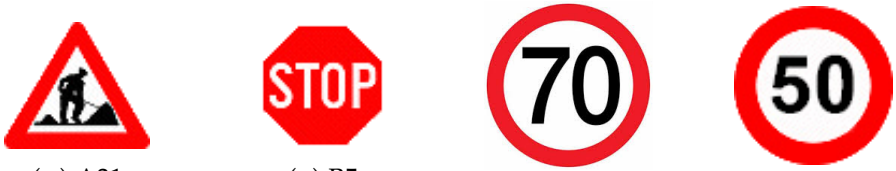
After a visual analysis of the images of the previously described database, we noted that there is an ambiguity of classification between the classes "Speed Limit 50" and "Speed Limit 70" which classed in the same class C43. So we divided this

latter into two classes C43_5 and C43_7.

Table 4.1 show us for each type of traffic sign the corresponding Class Id and the number of images.

Class Id	**Traffic Sign Type**	**Number of images**
B5	Stop Sign	69
A31	Construction Sign	78
C43_5	Speed Limit 50 Sign	86
C43_7	Speed Limit 70 Sign	99

TABLE 4.1: Data identification



(A) A31 (B) B5 (C) C43_7 (D) C43_5

FIGURE 4.13: Filtered Traffic Signs

Deep Learning models need a high number of data so the training results respond to our need. That's why we applied data augmentation to our data so it much the the need of the algorithm which are size and number of images. SSD Mobilenet-V2 (300×300) need images with size (300×300) if images are not correspond to this size so automatically this model applied a resize to each one. That's why we applied a data augmentation for the traffic signs data because we don't have too much data and the resolution of the images is very high. This data augmentation applied by cropping each image around the object in different positions (Figure 4.14). Table 4.2 show us the updated numbers of images after data augmentation.



FIGURE 4.14: Example of images after data augmentation

Class Id	Traffic Sign Type	Number of images
B5	Stop Sign	835
A31	Construction Sign	901
C43_5	Speed Limit 50 Sign	927
C43_7	Speed Limit 70 Sign	1100

TABLE 4.2: Data identification after data augmentation

Ground truth data for pedestrians was created using LabelImg which allow us to create a bounding box around each object and save the data as an XML file. The number of pedestrians data is still the same 352 images. Before starting the training phase, the following steps were carried out:

- Random division of data (images and labels) into training sets (90%) and test sets (10%).
- Converting XML files for pedestrians and TXT files for traffic signs together with a script to Data-Frame then to CSV were each annotation composed of 8 columns and the classes id transferred to the known name type(Figure 4.15). So we obtained in the final two CSV files, one for training and the other for testing.
- Creation of the label map files (with extension .pbtxt) which namely maps each of the used labels to an integer value. Figure 4.16 show us an example of the first two objects representing how this file got generated.
- Generation of two TFRecord[1] files for train and test using the previously generated CSV files and that by using the script from the TensorFlow object detection API.

	filename	width	height	class	xmin	ymin	xmax	ymax
0	image.001450_5_300_1250.jpg	300.0	300.0	Speed Limit 50	59.0	22.0	193.0	152.0
1	image.001450_5_300_1300.jpg	300.0	300.0	Speed Limit 50	9.0	22.0	143.0	152.0
2	image.001452_5_150_1000.jpg	300.0	300.0	Speed Limit 50	180.0	175.0	280.0	274.0
3	image.001452_5_150_1050.jpg	300.0	300.0	Speed Limit 50	130.0	175.0	230.0	274.0
4	image.001452_5_150_1100.jpg	300.0	300.0	Speed Limit 50	80.0	175.0	180.0	274.0

FIGURE 4.15: Data annotations

[1]TFRecord is a binary format for efficiently encoding long sequences so that TFRecord files easily loaded by TensorFlow

```
item {
    id: 1
    name: 'Construction'
}

item {
    id: 2
    name: 'Speed Limit 50'
}
```

FIGURE 4.16: Label map file example

Training and evaluation

For the choice of the model we used the pre-trained SSD Mobilenet-V2 (300 $\times$ 300) provided by tensorflow, because it is imposed by the hardware (Raspberry Pi + NCS2) [4].

We downloaded the pre-trained network for the model before the 1st July version because later commits lead to errors in the phase of deployment-related to NCS2. After the training of the model is finished, we extracted the newly trained inference graph from the obtained checkpoint to a binary file, which will be used to perform the validation and test. on the . Table 4.3 shows us the results of different loss functions and the mAP related to training and validation steps knowing that mAP is calculated with IoU superior to 0.5. Figure 4.18 show us the different validated results.

	Class_L	**Local_L**	**Regular_L**	**Total Loss**	**mAP**
Training	0.438	0.063	0.256	0.757	0.948
Validation	0.57	0.086	0.258	0.914	0.873

TABLE 4.3: Evaluation results

Results obtained in the total loss are ideal for both training and validation as suggested by the object detection API documentation [30] that the ideal total loss is lower than 1 and not close to 0. Figure 4.17 prove that the total loss results of training converged to lower than 1 in 40000 steps.

Of course, the model is not 100% ideal we can find sometimes errors in classification and localization. As obtained in our model and the validation phase, there is sometimes objects not detected which means the output images from the model are the same as the input or miss-detected like the represented figure 4.19.



FIGURE 4.17: Total loss graph



FIGURE 4.18: Example of validated test data



FIGURE 4.19: False detected person

Deployment and Test

After the trained and validated results of the model are perfectly realized as the showed previous table 4.3. We extracted the previous file to a binary file than we deployed it successfully into our embedded system after few trials.

The performance in this test step is calculated by gathering some images (74 images) from real video in which objects are present, then labeling this frames and finally by using the methods of calculating metrics offered by object detection API we got the following results: mAP with value of 0.823 and Total loss with 1.07. This prove to us that our model is trained successfully and the data is well preprocessed. Figure 4.20 show us different successful detection results by our robot using Raspberry Pi and NCS2.

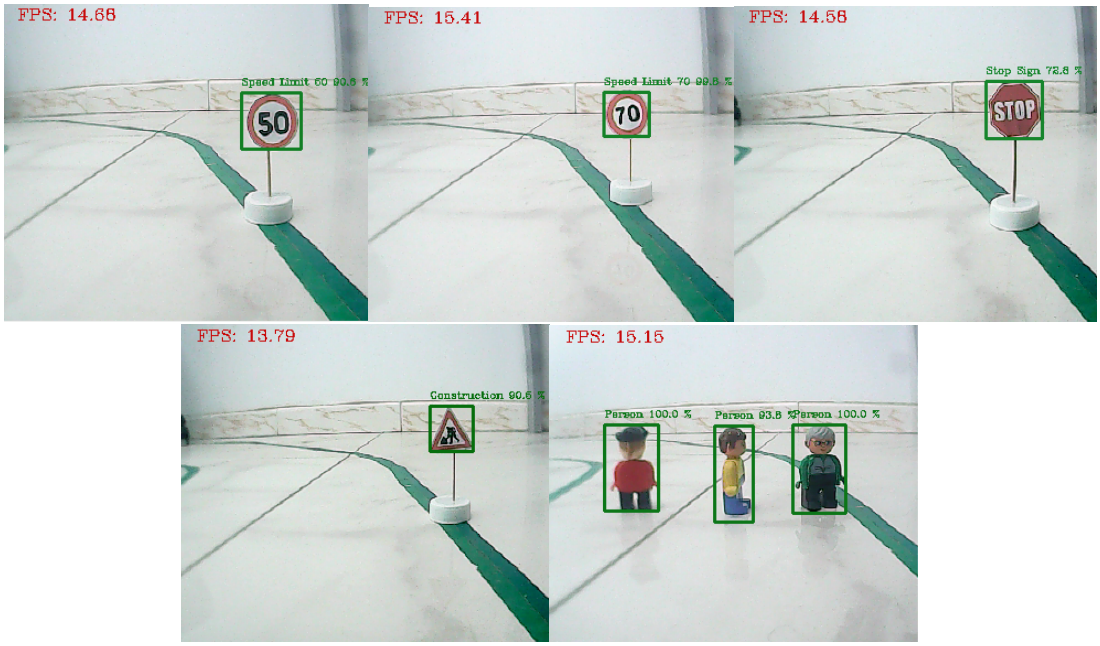


FIGURE 4.20: Example of real test data

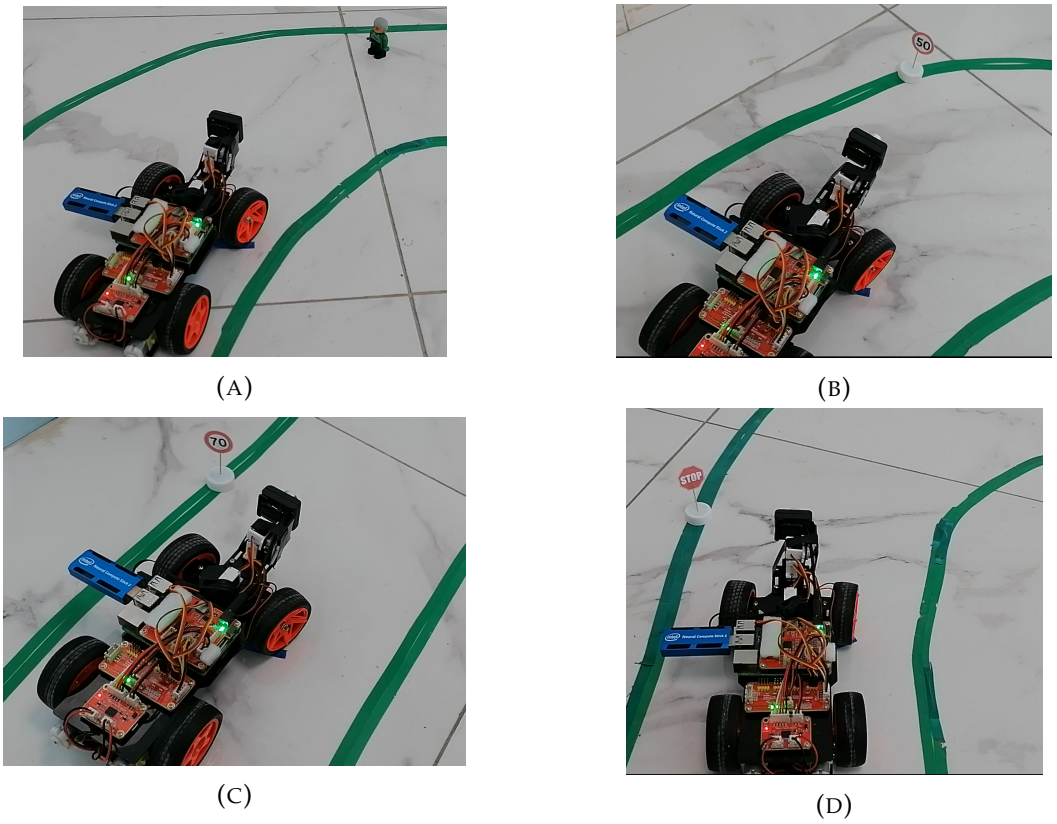
4.2.3 Throttle Control Method

After the detection process is succeeded, rule-based control is needed to take actions according to each detected object. So table 4.4 shows us the different values of the action throttle for each target knowing that throttle values are between 0 and 1.

Finally, The combination of previous sections gives us a successful autonomous driving, as shown in pictures (Figure 4.21) picked out from a demo video.

Object Type	**Throttle value**
Person	0
Stop sign	0
Construction sign	0.25
Speed limit 50 sign	0.5
Speed limit 70 sign	0.7

TABLE 4.4: Throttle values for each detected object



(A) (B) (C) (D)

FIGURE 4.21: Real robot navigation

4.3 Second Approach

The task of the agent is to navigate successfully and correctly for a determined number of time-steps along a given route so it can't move outside the lane lines. Furthermore, the agent is supposed to react on local objects like a person. As input, the robot gets a representation of its environment and interacts with it by controlling the throttle and the steering angle.

For each episode[2], the robot begins from the same point that it started at the beginning. Each episode successfully fulfilled if the time of the robot navigation without error reaches a fixed threshold of time-steps. Furthermore, the episode will be stopped, if the RL-agent drives into an obstacle (Person) or the difference between its coordinates and the path's centre exceed a certain threshold means that if the robot moves outside the lane.

4.3.1 Data Collection

As shown in Figures 4.22 and 4.23, different gathered data represented from reality and simulation. This data is composed of 20618 real and 18361 simulated images between lines and person together. The shape of all different images is the same (120, 160).

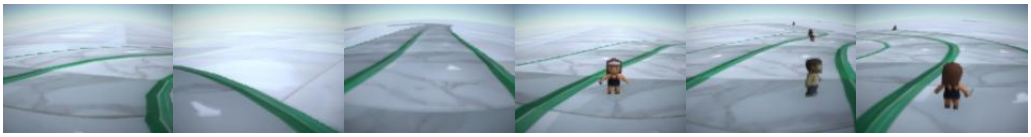


FIGURE 4.22: Example of simulated images



FIGURE 4.23: Example of real images

It is right that reinforcement learning doesn't need any data to learn, but it just needs the available environment, then it can handle the rest. So we discover together in the next step what this data offer to us even it is not concerned to train the RL model.

4.3.2 Simulation to Reality and State Representation Learning

There are three approaches to translate a trained robot from simulation to reality:

- **System identification :** System identification is to build a mathematical model for a physical system; in the context of RL, the mathematical model is the simulator. To make the simulator more realistic, careful calibration is necessary.

[2]Episode: Begin with the start of the robot and end when the car crashes into something or moved outside the line.

Unfortunately, calibration is expensive. Furthermore, many physical parameters of the same machine might vary significantly due to temperature, humidity, positioning or its wear-and-tear in time.

- **Domain randomization :** With domain randomization (DR), we can create a variety of simulated environments with randomized properties and train a model that works across all of them. Random properties take so much time to generate because we are using a unity simulator.

- **Domain adaptation :** Refers to a set of transfer learning techniques developed to update the data distribution in simulation to match the real one through mapping or regularization enforced by the task model. In other words, this is a translation between real and simulated images. Domain adaptation requires a decent amount of real data samples to capture the distribution.

The best approach that works for us is Domain adaptation because we have that amount of real data which is collected rapidly by the robot, so less time for preparation for the next step.
In our case Domain adaptation is Image-to-Image Translation[3] because we have only images from the front camera as input data. So the translation is realized between the real and simulated images.
So to make it clear, the RL-agent is trained in simulation with images coming from the front Webcam. The captured images are faded frame by frame at first to the trained image-to-image translation model, so that from the input simulated image, we get a real one. Then, the real image will be faded into the RL-algorithm so it can be trained as it is in the real world. So first we must choose the best model of image-to-image translation to respond to our needs. There are so many approaches that focus in this domain adaptation for example: Pix2Pix, Cycle GAN, UNIT, etc. All these models are a deep learning one with so many layers so we have big challenge to solve which is the execution time and available resources for training and testing. First our biggest concern is resources because if our resources can't support this main chall For resources, we have Google Colab for training For this aim, we used the open-source framework Unsupervised Image-to-Image Translation Networks (UNIT) [16] which is adapted. The collected data in the previous step is used for training a deep learning model.

First we trained the model as it is, we obtained good results after 90000 time step of training. After that we checked if the RL model can be trained in the simulation

[3]Image-to-image translation is the task of taking images from one domain and transforming them, so they have the style (or characteristics) of images from another domain.

with the generated vector in the latent space from simulated image. Unfortunately, we get very low fps approximately 1 which it can't train the RL model or test it

Table 4.5 show us the available network with latent space shape (512, 10, 10) which is bigger then the original image size so this can't be trained on the computer without powerful GPU nether tested in the robot because first if with CPU core i7 it can't be worked then raspberry pi + NCS2 also can't because they are less powerful then this CPU. So, the solution is to minimize this latent space so the robot can be trained and tested without any problems. After many trials of modifying and testing the UNIT architecture, we updated this network successfully as shown in table 4.6 which indicates the updated network that works successfully.

Encoders					
Layer	**Type**	**Activation**	**Size**	**Filter Size**	**Stride size**
1	Convolution	ReLU	64 Filter	7	1
2	Convolution	ReLU	128 Filter	4	2
3	Convolution	ReLU	256 Filter	4	2
4	Residual block	ReLU	512 Filter	3	1
5	Residual block	ReLU	512 Filter	3	1
6	Residual block	ReLU	512 Filter	3	1
7	Residual block	ReLU	512 Filter	3	1
Decoders					
Layer	**Type**	**Activation**	**Size**	**Filter Size**	**Stride size**
1	Residual block	ReLU	512 Filter	3	1
2	Residual block	ReLU	512 Filter	3	1
3	Residual block	ReLU	512 Filter	3	1
4	Residual block	ReLU	512 Filter	3	1
5	Convolution	ReLU	256 Filter	5	1
6	Convolution	ReLU	128 Filter	5	1
7	Convolution	TanH	3 Filter	7	1

TABLE 4.5: Network architecture for the unsupervised image-to-image translation

4.3.3 RL-Agent Setup and training

As Deep Reinforcement Learning approach, Soft Actor-Critic (SAC), described in chapter 2.5.1, is used. The implementation from the open source project stable-baselines [1] are applied. Stable-baselines provides a set of improved RL-algorithms like SAC, PPO, DQN, DDPG and more. Details about the general SAC training parameter settings can be found in the appendix in table C.1 and C.2. The deep learning

Encoders					
Layer	**Type**	**Activation**	**Size**	**Filter Size**	**Stride size**
1	Convolution	ReLU	32 Filter	7	1
2	Convolution	ReLU	64 Filter	4	2
3	Convolution	ReLU	128 Filter	4	2
3	Convolution	ReLU	256 Filter	4	2
4	Residual block	ReLU	512 Filter	3	1
5	Fully Connected	Linear	128 Neurons		
Decoders					
Layer	**Type**	**Activation**	**Size**	**Filter Size**	**Stride size**
1	Fully Connected	Linear	51200 Neurons		
2	Residual block	ReLU	512 Filter	3	1
3	Convolution	ReLU	256 Filter	5	1
4	Convolution	ReLU	128 Filter	5	1
5	Convolution	TanH	3 Filter	7	1

TABLE 4.6: Updated network architecture for the unsupervised image-to-image translation

part is realized in the Tensorflow framework [9] and allows to define custom policy-networks. Moreover, it is possible to plug in a custom simulation environment that needs to implement the OpenAI-Gym interface gym.Env which is our donkey car simulator.

The RL-agent needs to get all relevant information about the current state of the environment to be able to fulfil the task successfully. The raw data in our case is a vector of numbers obtained from the image-to-image translation latent space that represents this image the captured image by the front camera of the robot (the robot in real and simulation environment) with shape (120,160,3) then this image is fed to the Image-to-image translation method which gives us finally .

Training results are presented

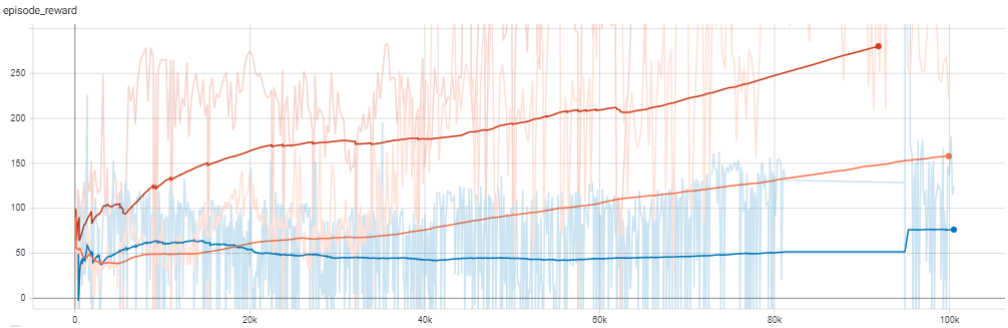


FIGURE 4.24: RL training graph evaluation for person crash avoidance

RL Algorithm	Training Time	Maximum Reward Value
SAC	1h 36m 56s	6654
DDPG	2h 3m 7s	1390
PPO	1h 37m 59s	226.6

TABLE 4.7: RL training results for person crash avoidance

4.3.4 Evaluation

Each algorithm is tested 5 times with 10000 steps in total which in each time different random track generated as well for the person lego. The gathered different results for each time are averaged and represented in the table 4.8. Number of crashing is simply the number of each time the simulated robot navigate out the track or crashed into pedestrian in 10000 steps. Mean Reward value is the averaged cumulative reward over 5 times.

RL Algorithm	Mean Reward Value	Number of crashing
SAC	2152.01	0
DDPG	1989.71	9
PPO	1855.74	26

TABLE 4.8: RL testing results for person crash avoidance

Our aim is not only making the most efficient model but also a safer one. So if all our algorithms do not work well in the simulation like PPO and DDPG in which many crashes occurred, there is no point to try these dangerous approaches in the real world even if they will perform better but actually not. So PPO and DDPG are not concerned for the next real evaluation step, so only SAC is the chosen one because it is the best model and 100% safe as shown in 4.8.

4.4 Final Evaluation

After developing and testing these two approaches we need to evaluate them between each other so we can in the future pick the appropriate method for our needs. The robot is tested for 20 rounds in each approach starting in different positions for every round, knowing that person legos placed randomly in the track. Table 4.9 show us the different parameters to evaluate between those two approaches.

Person crash is the number of collisions between the robot and the lego. Line crash is the number of times that the robot navigates out the track. Time is the time of driving for 20 rounds.

The obtained results confirm that Reinforcement Learning approach which is a self-learning method is much better than the rule-based control method that need human intervention.

Approach type	**Lane crash**	**Time**
First Approach	7	14 min
Second Approach	2	12 min 21 s

TABLE 4.9: Evaluation results between first and second approach for lane detection

Approach type	**person crash**
First Approach	0
Second Approach	0

TABLE 4.10: Evaluation results between first and second approach for person detection

4.5 Conclusion

In this chapter we have approached all the experimental aspects of this project, we have started by detailing the stages of the implementation of each approach separately, these stages have been highlighted through screenshots of the tests performed, and we have completed the chapter with a final evaluation which proves that self-learning approach better than the hand-coded approach.

Conclusion

As part of the graduation internship achieved within SAGEMCOM, we worked on implementing two AI-based solutions for autonomous driving cars. More particularly, we managed to prepare a fully functional prototype of a self-driving robot using PiCar-V and an AI-based controlling system.

The achieved work serves as an experimental POC for more sophisticated and ready-to-market self-driving systems.

All along, we worked on two principal axes. we first implemented a computer vision-based solution, following up with other self-driving car manufacturers (As it's the most common choice on the market). This approach uses image processing and deep learning to detect and identify objects and upon that the system determines how the car needs to interact with its environment. The second approach uses the reinforcement learning paradigm to build the model that takes actions in an environment in a way that maximize the notion of cumulative reward set in correlation with the outer environment.

This project is setting the experimental bases for a more advanced system that uses more adequate detecting sensors and more performant calculating systems implemented on real-world cars.

Further modifications and enhancements need to be conducted especially in the reinforcement learning approach as the experimental results show its advantages both in terms of cost and performance. for instance, full integration of object detection needs to be done (detecting the traffic signs), the process of integration needs to start with the simulation and then verifying the model in the real-world environment.

Appendix A

Hardware configuration

Raspberry Pi and NCS2 configuration

To configure the Raspberry Pi and the NCS2 and adapt them to the needs of this project, we applied the following steps:

- Installation of the Raspberry Pi OS with NOOBS [6] on a 32GB microSD card.
- Insertion of the microSD card into the Raspberry Pi and booting it up.
- Entering the WiFi credentials and enabling SSH, VNC, and the camera interface and that's via HDMI screen + keyboard/mouse for the initial WiFi configuration.
- Rebooting then connecting via VNC.
- Updating the system and installation of the OpenCV dependencies (developer tools including CMake, a selection of image and video libraries, GTK, packages which contain numerical optimizations for OpenCV, Python 3 development headers)
- Downloading the OpenVINO toolkit version 2020.3.22 (other versions doesn't work well) and installing it.
- Adding the current user to the Raspbian "users" group and Configuring USB rules by loading install_NCS_udev_rules.sh.
- Creation of a python virtual environment to hold OpenVINO, OpenCV and related packages.

Appendix B

Lane detection parameters

Parameter	Value
lower_green_hsv	(60, 70, 70)
upper_green_hsv	(90, 255, 255)
canny	(200, 400)
precision	1
angle	1
$min_threshold$	12
minLineLength	11
maxLineGap	5

TABLE B.1: Parameters applied in lane detection method

Appendix C

RL setup and training

Used Parameters in RL Algorithm

Parameter Name	Value
Action repeat	1
Image size	(120, 160, 3)
Max CTE error	1.8
Max steering	1
Max steering difference	0.1
Max throttle	0.5
Min throttle	0
Reward crash person	-40
Reward crossing error	-30
Throttle reward weight	0.1
VAE vector size	128
Weight reward crash speed	15

TABLE C.1: Common Parameters of Different RL Algorithms

Parameter Name	Value
Learning rate	3e-4
Buffer size	30000
Batch size	64
Gamma	0.99
Entropy coefficient	auto_0.1
Gradient steps	600
Learning starts	300

TABLE C.2: SAC parameters

RL training results for lane navigation

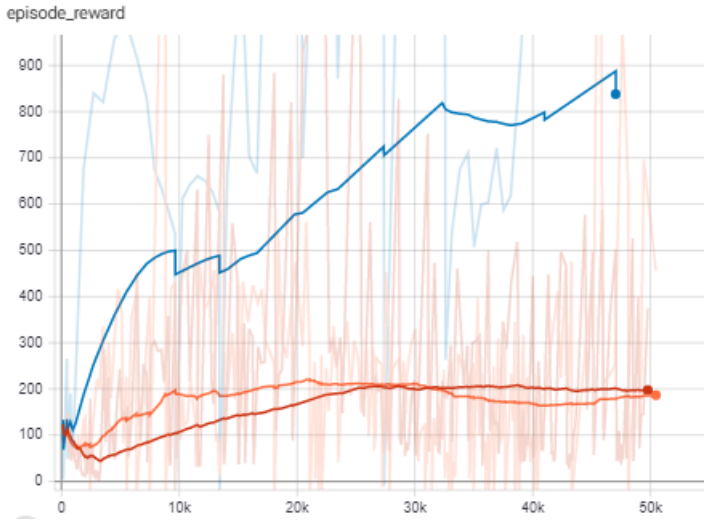


FIGURE C.1: RL training graph evaluation for lane navigation

RL Algorithm	Training Time	Maximum Reward Value
SAC	44m 26s	6326
DDPG	1h 1m 19s	1505
PPO	46m 57s	2007

TABLE C.3: RL training results for lane navigation

Appendix D

Additional equations

Batch Normalisation

Batch normalization applies normalization over the whole batch by zero-centering and rescaling the data. It is expected that the mean of the normalized data is close to zero and the variance is close to one. In a batch H with m samples, each value h_i is normalized over the whole batch according to equation D.2. The mean μ (equation D.3) and variance σ (equation D.4) is computed element-wise for each spatial position across the whole batch. $\delta > 0$ is a small value to avoid division by zero. The normalized value $y_i\prime$ will be further processed by equation D.1. γ_{bn} and α_{bn} are parameters of the batch normalization (b_n) layer and are learned along with the original parameter set θ of the Neural Network. The additional learning dynamics increase the network's expressive power.

$$y_i = \gamma b_n y_i\prime + \beta_{bn} \tag{D.1}$$

$$y_i\prime = \frac{h_i - \mu}{\sigma} \tag{D.2}$$

$$with \;\; \mu = \frac{1}{m}\sum_{i=1}^{n} h_i \tag{D.3}$$

$$with \;\; \sigma = \sqrt{\frac{1}{m}\sum_{i=1}^{m}(h_i - \mu)^2 + \delta} \tag{D.4}$$

Applying the batch normalization to the input data as well as the output of any hidden layers leads to a regularizing effect during learning and prevents overfitting. Another advantage is that it speeds up the training time. It has to be noted that batch normalization is only applicable, if the exact position of the features is not relevant, but rather whether the feature exists in the input.

Bellman equation

$$\begin{aligned} v_\pi(s) &= \mathbb{E}_\pi[G_t|S_t = s] \\ &= \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1}|S_t = s] \\ &= \sum_a \pi(a|s) \sum_{s\prime} P^a_{s,s\prime}[R^a_{s,s\prime} + \gamma \mathbb{E}_\pi[G_{t+1}|S_{t+1} = s\prime]] \\ &= \sum_a \pi(a|s) \sum_{s\prime} P^a_{s,s\prime}[R^a_{s,s\prime} + \gamma v_\pi(s\prime)] \end{aligned} \tag{D.5}$$

Monte Carlo Method

The Monte Carlo method is an approach that aims to solve reinforcement problems with episodic tasks, where no model of the environment exists. It is an iterative approach and converges with the increasing number of episodes towards the optimal policy. There is a Q-table that holds the action-value for each possible state-action pair. The value is the average over all returns, that has been collected in all episodes. An entry of the table is updated each time a state-action pair is met by the agent. The update equation is shown in equation D.6, where N(S_t , A_t) is the number of visits of the state-action pair (S_t , A_t).

$$Q(S_t, A_t) = Q(S_t, A_t) + \frac{1}{N(S_t, A_t)}(G_t - Q(S_t, A_t)) \tag{D.6}$$

The different returns can be weighted by . Instead of taking the true average, recent returns can be weighted more or less strongly (see equation D.7).

$$\begin{aligned} Q(S_t, A_t) &= Q(S_t, A_t) + \alpha(G_t - Q(S_t, A_t)) \\ &= (1 - \alpha)Q(S_t, A_t) + \alpha G_t \end{aligned} \tag{D.7}$$

The Monte Carlo method iterates over episodes. During the Evaluation step, the agent acts according to policy π for one episode. When the episode is finished, the agent collected a sequence of experiences S_1, A_1, R_2, S_2, ..., S_T and can update the Q-table according to it. For each state-action pair (S_t , A_t) in the sequence, the expected return G_t is retrieved and the corresponding entry in the Q-table is updated according to equation D.7. During the Improvement step, the policy π will be updated according to the recent Q-table. In the next iteration, the Evaluation step is performed with the new, updated policy. Figure D.1 illustrates the concept of the Monte Carlo method.

The most conservative policy update is the so called greedy policy. The agent always chooses the action with the maximum action-value. Greedy actions exploit the current knowledge; the agent can get stuck and end up in a non-optimal policy.

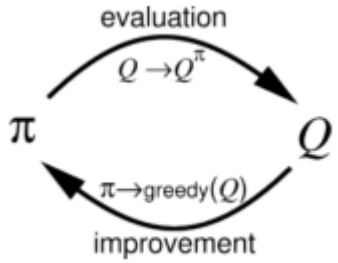


FIGURE D.1: Concept of the Monte Carlo method

To avoid that situation, explorative actions should be taken from time to time. A policy is called ϵ-greedy policy, if the greedy action is taken with a probability of (1ϵ) and a random action is taken with a probability of ϵ to explore the action space.

ϵ is a value between 0 and 1 and weighs the relation between exploitation and exploration. It stands to reason to explore the action space stronger in the beginning because the agent does not have reliable knowledge yet. The more experiences the agent gains, the more learned knowledge should be exploit. An ϵ that decreases over time to a minimum value of ϵ_{min} models that behavior.

Sarsa

Sarsa is an on-policy TD control method and stands for S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1}. Those parameters are needed in each iteration to update the action-value Q(s, a) of the state-action (S_t, A_t) in the Q-table. The agent takes action a in state s and receives the immediate reward R_{t+1}. Afterwards policy π is used to determine action A_{t+1}, that will be taken in the next state. The sum of the immediate return R_{t+1} and the Q-value of the next state-action pair S_{t+1}, A_{t+1} represents the TD-target and approximates the true expected return G_t. The action-value update rule is shown in equation D.8.

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)] \tag{D.8}$$

Deep Q-Network (DQN)

Combining Q-Learning with non-linear function approximation has been investigated in the past decades and did not lead to great success because of unstable learning. In 2015, the DeepMind group presented an approach – called deep Q-Network (DQN) – that showed a great success. They combined the model-free, off-policy Q-Learning with Deep Neural Networks. As input data, high-dimensional raw sensory

input with no previously hand-crafted features are used. This end-to-end architecture allows the network to extract relevant features by itself. The output of the Q-network provides a probability distribution over all possible discrete actions. This allows one to determine the best action for a given state with a single forward pass. Particularly, the problem of unstable learning has been improved with two additional mechanisms, called experienced replay and frozen target network, that will be further explained in the following.

The idea of experienced replay is to store the agent's experiences S_t, A_t , R_{t+1}, S_{t+1} in a buffer that can hold n_{buf} experiences in total. In each training step, a batch of experiences is uniformly sampled from the buffer and fed to the network. Experienced replay removes the correlations in the data sequences and feeds the network with independent data. It also ensures that old experiences are repeated from time to time. It has a smoothing effect over changes in the data distribution.

In DQN the network is updated according to the loss function from equation D.9. The loss function is computed by taking the squared TD-error.

$$L_i(\theta_i) = \mathbb{E}_t[(R_{t+1} + \gamma \max_a Q(S_{t+1}, a, \theta_i^-) - Q(S_t, A_t, \theta_i))^2] \tag{D.9}$$

In equation D.9, the second mechanism frozen target network is introduced as well. Two networks with the same structure, but different weights are used: θ for the Q-network and θ^- for the target network. The Q-network is regularly updated according to the loss function from equation 2.29, while the target network is updated by copying the parameters of the Q-network to the target network $\theta^- = \theta$ every C time steps. Thus, the weights of the target network θ^- are held frozen for C time steps. It smooths oscillating policies and leads to more stabilized learning.

REINFORCE algorithm

The REINFORCE algorithm is one of the first and simplest Policy Gradient Method introduced in 1992. In the following, a variant of the original algorithm will be presented to demonstrate the key procedure of a Policy Gradient Method. A trajectory τ is defined as a state-action sequence with the length T: S_0, A_0, S_1, A_1, ...S_T, A_T, S_{T+1}. The difference between a trajectory and an episode is that the last state of a trajectory does not need to be final. The policy performance measure J(θ) in equation D.10 is defined by the expected return of all trajectories τ. The contribution of each trajectory τ to the expected return is the product of the cumulative reward R(τ) and

the probability of its occurrence $\pi_\theta(\tau)$ under policy π_θ.

$$J(\theta) = \mathbb{E}[\sum_{\tau} R(\tau)\pi_\theta(\tau)] \tag{D.10}$$

The derivative of J(θ) can be determined by applying the Policy Gradient Theorem. This results in equation D.11.

$$\nabla J(\theta) = \mathbb{E}[\sum_{\tau} R(\tau)\nabla_\theta \log \pi_\theta(\tau)] \tag{D.11}$$

The REINFORCE algorithm approximates $\nabla J(\theta)$ with equation D.12. Only one trajectory $\tau^{(i)}$ is used per iteration i to approximate the gradient of the objective function. Instead of using the raw cumulative, discounted reward R(τ), the advantage Adv$^{\pi_\theta}$ (S_t, A_t) is used. It compares the true, cumulative, discounted reward $\sum_{k=0}^{T-t} \gamma^k R_{t+k}$ to the expected return $V^\pi(S_t)$, that is estimated by a Critic network. If the true return is higher than the expected return, the advantage is positive, and the Actor will be updated in such a manner, that it is more likely to choose action A_t in state S_t. If the true return is smaller than the expected reward, the probability of taking action A_t in state S_t will be decreased.

$$\begin{aligned} \nabla_\theta J(\theta) \approx \hat{g} &= \sum_{t=0}^{T} Adv^{\pi_\theta}(S_t, A_t)\nabla_\theta \log \pi_\theta(A_t|S_t) \\ with \;\; Adv^{\pi_\theta}(S_t, A_t) &= \sum_{k=0}^{T-t} \gamma^k R_{t+k} - V^\pi(S_t) \end{aligned} \tag{D.12}$$

Finally, the approximated gradient $\hat{g}$ is used to apply gradient ascent to update the policy parameters θ.

$$\theta \leftarrow \theta + \alpha\hat{g} \tag{D.13}$$

Webography

[1] M. Ernestus A. Gleave A. Hill A. Raffin and A. Kanervisto. *Stable Baselines*. Online; accessed 19 Sept 2020. URL: https://github.com/hill-a/stable-baselines.

[2] Anaconda. *Anaconda cloud*. Online; accessed 20 Sept 2020. URL: https://www.anaconda.com/.

[3] Intel Corporation. *NCSDK Documentation*. Online; accessed 17 Sept 2020. URL: https://movidius.github.io/ncsdk/.

[4] Nvidia Developer. *Jetson Nano: Deep Learning Inference Benchmarks*. Online; accessed 04 Oct 2020. URL: https://developer.nvidia.com/embedded/jetson-nano-dl-inference-benchmarks.

[5] Facebook. *FROM RESEARCH TO PRODUCTION*. Online; accessed 20 Sept 2020. URL: https://pytorch.org/.

[6] RASPBERRY PI FOUNDATION. *NOOBS*. Online; accessed 04 Oct 2020. URL: https://www.raspberrypi.org/downloads/noobs/.

[7] Brian Fung. *The technology behind the Tesla crash, explained*. Online; accessed 04 Oct 2020. URL: https://www.washingtonpost.com/news/the-switch/wp/2016/07/01/the-technology-behind-the-tesla-crash-explained/.

[9] Google. *Tensorflow*. Online; accessed 19 Sept 2020. URL: https://www.tensorflow.org/.

[11] Leigh Johnson. *OpenAI Gym Environments for Donkey Car*. Online; accessed 20 Sept 2020. URL: https://gym-donkeycar.readthedocs.io/en/latest/.

[12] A. Karpathy. *CS231n Convolutional Neural Networks for Visual Recognition*. https://cs231n.github.io/understanding-cnn/.

[15] Microsoft. *Visual Studio Code*. Online; accessed 20 Sept 2020. URL: https://code.visualstudio.com/.

[17] OpenAI. *Learning Dexterity*. Online; accessed 04 Oct 2020. URL: https://openai.com/blog/learning-dexterity/.

[18] ISL Products. *DESIGN NOTE: DC MOTOR / DC GEAR MOTOR BASICS*. Online; accessed 21 Sept 2020. URL: https://www.islproducts.com/design-notes/dc-motor-dc-gear-motor-basics/.

[19] Google Research. *Colaboratory*. Online; accessed 04 Oct 2020. URL: https://research.google.com/colaboratory/faq.html.

[20] DIY Robocars. *An opensource DIY self driving platform for small scale cars*. Online; accessed 20 Sept 2020. URL: http://docs.donkeycar.com/.

[22] SunFounder. *PCA9685 16 Channel 12 Bit PWM Servo Driver*. Online; accessed 21 Sept 2020. URL: http://wiki.sunfounder.cc/index.php?title=PCA9685_16_Channel_12_Bit_PWM_Servo_Driver.

[23] SunFounder. *SF006C Clutch Gear Digital Servo*. Online; accessed 21 Sept 2020. URL: http://wiki.sunfounder.cc/index.php?title=SF006C_Clutch_Gear_Digital_Servo.

[24] SunFounder. *SunFounder Smart Video Car Kit for Raspberry Pi*. Online; accessed 20 Sept 2020. URL: https://github.com/sunfounder/SunFounder_PiCar-V.

[25] SunFounder. *TB6612 DC Motor Driver Module*. Online; accessed 21 Sept 2020. URL: http://wiki.sunfounder.cc/index.php?title=TB6612_DC_Motor_Driver_Module.

[28] Radu Timofte. *BelgiumTS Dataset*. Online; accessed 04 Oct 2020. URL: https://btsd.ethz.ch/shareddata/index.html.

[29] Stanford University. *Neural Networks Part 1: Setting up the Architecture*. Online; accessed 19 Sept 2020. URL: https://cs231n.github.io/neural-networks-1/.

[30] Lyudmil Vladimirov. *Training Custom Object Detector*. Online; accessed 04 Oct 2020. URL: https://tensorflow-object-detection-api-tutorial.readthedocs.io/en/latest/training.html.

[32] Kostiantyn Isaienkov Yaroslav Isaienkov and Alina Zhuravel. *LEGO Minifigures*. Online; accessed 04 Oct 2020. URL: https://www.kaggle.com/ihelon/lego-minifigures-classification.

Bibliography

[8] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. http://www.deeplearningbook.org. MIT Press, 2016.

[10] P. Dhariwal A. Radford J. Schulman F. Wolski and O. Klimov. *Proximal Policy Optimization Algorithms*. http://arxiv.org/abs/1707.06347. CoRR, 2017.

[13] Yann Lecun et al. *Gradient-based learning applied to document recognition*. 1998, pp. 2278–2324.

[14] Andrew Markham Linhai Xie Sen Wang and Niki Trigoni. *Towards Monocular Vision based Obstacle Avoidance through Deep Reinforcement Learning*. Online; accessed 8 Sept 2020. URL: https://arxiv.org/pdf/1706.09829.pdf.

[16] Jan Kautz Ming-Yu Liu Thomas Breuel. *Unsupervised Image-to-Image Translation Networks*. Online; accessed 04 Oct 2020. URL: https://arxiv.org/abs/1703.00848.pdf.

[21] S. Shalev-Shwartz and S. Ben-David. *Understanding machine learning: From theory to algorithms*. https://www.cs.huji.ac.il/~shais/UnderstandingMachineLearning/. Cambridge university press, 2014.

[26] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*. http://incompleteideas.net/book/RLbook2018.pdf. MIT Press, 2018.

[27] P. Abbeel S. Levine T. Haarnoja A. Zhou. *Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor*. http://arxiv.org/abs/1801.01290. CoRR, 2018.

[31] Lilian Weng. *Object Detection Part 4: Fast Detection Models*. 2018. URL: http://lilianweng.github.io/lil-log/2018/12/27/object-detection-part-4.html.